

LEARNING HUB V3 — COMPLETE AS-BUILT FINAL SOFTWARE PRODUCT DOCUMENTATION

Document Type: Final As-Built Software Product Documentation Suite

Status: Production As-Built Specification (Verified against Application & Codebase)

<!-- As_Built_Product_Documentation.md -->

AS-BUILT PRODUCT DOCUMENTATION — LEARNING HUB V3

Executive Summary: "What Exactly Did We Build?"

Learning Hub V3 is an enterprise Learning & Development (L&D) platform engineered to manage, deliver, track, and certify corporate and academic training programs. Built to scale to 60,000+ active learners, the product unifies self-paced digital courseware, live online sessions, and campus-based in-person workshops into a single responsive web system.

1. Vision Comparison: Intended vs. Actually Implemented

Domain	Reconstructed Intended Vision	Actually Implemented System	Status / Gap
Learner Authentication	Enterprise Single Sign-On (SSO / Google OAuth2)	Passwordless Global ID login (/learner/login)	Gap: Password verification not implemented; passwordless lookup.
Multi-Mode Courses	Self-Paced, Live Online, Live In-Person	Fully supported with distinct course ID prefixes (CRS-SP-, CRS-ON-, CRS-IP-)	Fully Implemented
Attendance Verification	Real-time QR scanning & manual audit	Mobile camera QR scan endpoint and manual override with mandatory reason logging	Fully Implemented
Courseware Authoring	Video, PDF, SCORM, Rise 360 block builder	Integrated SCORM player, pypdfium2 PDF parser, pptx renderer, and Rise 360 versioned block builder	Fully Implemented

Certification	Automated verifiable PDF certificates	Instant ReportLab PDF generation with unique CERT-UUID and public verification portal (/certificates/verify)	Fully Implemented
Scalable Storage	Decoupled S3 / Backblaze B2 / MinIO storage	b2_service.py with boto3 AWS S3 compatibility for low-cost media delivery	Fully Implemented
Automated Testing	Comprehensive CI/CD test regression suite	0 automated unit/integration test files found	Gap: 0% test coverage.

2. Final System Architecture Summary

```
flowchart LR
  LearnerClient[Learner Mobile / Desktop] --> FlaskApp[Flask Monolith Server]
  AdminClient[Admin Web Browser] --> FlaskApp

  subgraph FlaskApp["Flask Monolith (13 Blueprints)"]
    Routes[Routes & Views]
    Services[SCORM / PDF / QR / Storage Services]
  end

  FlaskApp --> DB[(SQLite WAL / PostgreSQL)]
  Services --> S3Storage[(Backblaze B2 / MinIO S3 Bucket)]
  Services --> LocalStorage[(Uploads Directory)]
```

3. Final Feature Matrix Summary

- **Total Implemented Features:** 20 Core Feature Modules
- **Total API Routes:** 40 Endpoints across 13 Blueprints
- **Total Database Models:** 14 Relational Models (29 SQL Tables)
- **Primary Tech Stack:** Python 3.14, Flask 3.1.3, SQLAlchemy 2.0, Jinja2, Bootstrap 5, Waitress WSGI, ReportLab, pypdfium2, boto3.

4. Key Recommendations for Future Maintenance

1. **Implement Learner Password / SSO Security:** Replace passwordless Global ID login with Google OAuth2 or bcrypt password validation.
2. **Build Automated Test Suite:** Implement [pytest](#) suite for core services to ensure zero regression failures.
3. **Refactor Route Controllers:** Split [courses.py](#) (100 KB) and [learners.py](#) (70 KB) into modular service controllers.

<!-- SRS.md -->

Learning Hub V3 — Software Requirements Specification (SRS)

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document defines the functional and non-functional requirements of **Learning Hub V3**, based strictly on the current implemented codebase.

1.2 Scope

Learning Hub V3 is an enterprise Learning & Development management web application designed for corporate and educational organizations. It supports multi-mode course delivery (Self-Paced, Live Online, Live In-Person), QR-code based live session attendance, interactive quizzes, automated PDF certificate generation, Rise 360 block content authoring, SCORM package execution, gamified engagement, social learning wall interactions, and administrative report exports.

1.3 Definitions and Terminology

- **Global ID:** Unique learner identification string (e.g. [10001](#)) used for passwordless login and corporate roster mapping.
- **Rise 360 Block:** Modular courseware JSON content schema allowing interactive accordion, flipcard, process, and quiz blocks.
- **SCORM:** Sharable Content Object Reference Model standard package ([.zip](#)) parsed and served by the platform.
- **WAL Mode:** Write-Ahead Logging database mode configured in SQLite for high-concurrency read/write operations.

2. System Overview

Learning Hub V3 follows a modular Flask architecture:

- **Presentation Layer:** Responsive HTML5 templates rendered server-side via Jinja2, styled with vanilla CSS, custom theme skins, and Bootstrap 5.
- **Application Layer:** 13 domain-specific Flask Blueprints managing business logic, routing, authentication, and file processing.
- **Persistence & Storage Layer:** SQLAlchemy ORM backing SQLite ([lms.db](#)) or PostgreSQL ([DATABASE_URL](#)), with hybrid local/S3-compatible object storage.

3. User Roles & Access Control Matrix

Role	Role Description	Accessible Modules	Key Restrictions
Super Administrator	Platform DevOps & DB Maintenance	/super_admin , /dashboard , all admin modules	Full unrestricted system access

L&D Administrator	Curriculum & Operations Manager	/courses , /classes , /learners , /attendance , /feedback , /certificates , /reports , /quizzes	Cannot modify system-level S3 configs or trigger DB wipes unless in super admin
Learner	Student / Employee	/learners/portal , /learning_wall , /certificates/verify , course flows	Cannot access admin interfaces or view other learners' private reports
Facilitator	Session Instructor	Live class roster view, QR seating display, attendance marking	Access limited to assigned live classes
Manager	Subordinate Manager	Learner portal team tab	Can only view direct reports (manager_id foreign key)

4. Functional Requirements

FR-001: Admin Authentication

- **Description:** L&D Administrators authenticate via username and password.
- **Actor:** L&D Administrator
- **Preconditions:** User navigates to [/login](#).
- **Main Flow:** Admin inputs username/password → system verifies hash with `AdminUser.password_hash` → session variable `admin_logged_in=True` set → redirect to [/dashboard](#).
- **Validation:** Username and password required.
- **Error Conditions:** Invalid credentials display error banner "Invalid Username or Password".
- **Implementation Status:** Fully Implemented

FR-002: Passwordless Learner Authentication

- **Description:** Learners log in using their Global ID without requiring a password.
- **Actor:** Learner
- **Preconditions:** Learner navigates to [/learner/login](#).
- **Main Flow:** Learner submits Global ID → system matches `Learner.global_id` → updates login streak and awards daily points → sets `session['learner_id']` → redirects to [/learners/portal](#).
- **Validation:** Global ID string required.
- **Error Conditions:** Non-existent Global ID returns "Learner with Global ID X not found".
- **Implementation Status:** Fully Implemented (Security weakness: passwordless).

FR-003: Course Creation & Management

- **Description:** Admins create, edit, archive, and delete courses.
- **Actor:** L&D Administrator

- **Preconditions:** Admin is logged in.
- **Main Flow:** Admin accesses [/courses/create](#) → fills details (Title, Duration, Mode: Self Paced / Live Online / Live In Person, Pass Percentage, Thumbnail) → system auto-generates ID ([CRS-SP-001](#), [CRS-ON-001](#), [CRS-IP-001](#)) → saves [Course](#) record.
- **Validation:** Pass percentage 0-100%, required title and mode.
- **Implementation Status:** Fully Implemented

FR-004: Course Lesson & Courseware Management

- **Description:** Admins add lessons and attach multi-format courseware (Video URL, PDF, PPT, Text, SCORM, Google Drive).
- **Actor:** L&D Administrator
- **Preconditions:** Course exists.
- **Main Flow:** Admin selects course → adds lesson (number, title, min_time_minutes) → uploads file or enters external URL → saves [LessonCourseware](#).
- **Implementation Status:** Fully Implemented

FR-005: Interactive Rise 360 Courseware Authoring

- **Description:** Admins author interactive block-based courseware versioned in JSON.
- **Actor:** L&D Administrator
- **Preconditions:** [ENABLE_CONTENT_AUTHORING=True](#) in environment.
- **Main Flow:** Admin accesses Rise editor for courseware → adds text, accordion, flipcard, process, or quiz blocks → saves version to [RiseCoursewareVersion](#) → publishes.
- **Implementation Status:** Fully Implemented

FR-006: SCORM Package Upload & Execution

- **Description:** Admins upload SCORM [.zip](#) packages; learners execute them via an embedded iframe.
- **Actor:** Admin / Learner
- **Preconditions:** SCORM zip package uploaded.
- **Main Flow:** Admin uploads SCORM zip → [scorm_service.py](#) extracts manifest [imsmanifest.xml](#) → finds launch HTML → learner launches courseware → iframe renders package.
- **Implementation Status:** Fully Implemented

FR-007: Live Class Scheduling & Roster Management

- **Description:** Admins schedule live classes linked to courses with facilitators and rosters.
- **Actor:** L&D Administrator
- **Preconditions:** Course and Facilitator learner exist.
- **Main Flow:** Admin navigates to [/classes/create](#) → selects course, mode (In Person/Online), date, location/meet_link, facilitator → system generates [CRS-CLS-XXXXXX](#) → admin enrolls learners.
- **Implementation Status:** Fully Implemented

FR-008: Real-Time QR Code Attendance

- **Description:** Facilitators display a class QR code; learners scan to mark attendance.

- **Actor:** Facilitator / Learner
- **Preconditions:** Live class active; learner logged in on mobile.
- **Main Flow:** Facilitator opens QR seating view → Learner scans QR URL `/attendance/scan/<class_id>` → system records `Attendance(status='Present', recorded_via='QR')`.
- **Implementation Status:** Fully Implemented

FR-009: Manual Attendance Override & Audit Logging

- **Description:** Admins manually update learner attendance status with mandatory reason logging.
- **Actor:** L&D Administrator
- **Preconditions:** Live class roster displayed.
- **Main Flow:** Admin changes status to Present/Absent/Late → submits mandatory justification reason → system updates `Attendance` and creates `AuditLog` entry.
- **Implementation Status:** Fully Implemented

FR-010: Assessment & Quiz Engine

- **Description:** System delivers Pre, Post, and Lesson MCQ assessments and calculates pass percentage.
- **Actor:** Learner
- **Preconditions:** Learner enrolled in course.
- **Main Flow:** Learner submits assessment answers → system evaluates correct options → computes percentage → records `AssessmentAttempt` → if score $\geq$ `course.pass_percentage`, marks passed.
- **Implementation Status:** Fully Implemented

FR-011: Automated PDF Certificate Generation

- **Description:** System generates verifiable PDF certificates upon course completion and feedback submission.
- **Actor:** System / Learner
- **Preconditions:** Course completed and feedback submitted.
- **Main Flow:** System checks eligibility → invokes `pdf_service.py` using ReportLab → generates PDF file `CERT-XXXXXX.pdf` → saves `Certificate` record → learner downloads.
- **Implementation Status:** Fully Implemented

FR-012: Certificate Verification Portal

- **Description:** Public endpoint to verify certificate authenticity by Certificate ID.
- **Actor:** Public / Employer
- **Preconditions:** Certificate ID provided.
- **Main Flow:** User visits `/certificates/verify?id=CERT-XXXXXX` → system queries database → displays learner name, course title, and issue date.
- **Implementation Status:** Fully Implemented

FR-013: Feedback Survey Repository & Submission

- **Description:** Admins build reusable feedback questionnaires; learners complete surveys after classes/courses.

- **Actor:** Admin / Learner
- **Preconditions:** Feedback repository attached to course/class.
- **Main Flow:** Learner completes course → redirected to feedback survey → submits responses → saved as JSON in [FeedbackResponse](#).
- **Implementation Status:** Fully Implemented

FR-014: Gamification Engine (Streaks, Points, Badges)

- **Description:** System tracks daily login streaks, awards points for activities, and grants badges.
- **Actor:** Learner / System
- **Preconditions:** Learner logs in or completes action.
- **Main Flow:** Learner logs in on consecutive days → streak increments → points awarded via [award_points\(\)](#) → badge awarded via [award_badge\(\)](#) when criteria met (e.g. 5-day streak).
- **Implementation Status:** Fully Implemented

FR-015: Social Learning Wall

- **Description:** Interactive wall for announcements, birthday wishes, course completion posts, reactions, and comments.
- **Actor:** Learner / Admin
- **Preconditions:** User logged in.
- **Main Flow:** System or user creates post ([LearningWallPost](#)) → users add reactions ([like](#), [love](#), [celebrate](#), [clap](#), [fire](#)) or submit text comments.
- **Implementation Status:** Fully Implemented

FR-016: Support Ticket Management

- **Description:** Learners log support tickets; admins review and resolve them.
- **Actor:** Learner / Admin
- **Preconditions:** Learner logged in.
- **Main Flow:** Learner submits issue description & category → [LmsIssue](#) created ([Open](#)) → Admin reviews in [/learners/issues](#) → marks [Resolved](#).
- **Implementation Status:** Fully Implemented

FR-017: Learner Bulk Import & Export

- **Description:** Admins import learners via Excel/CSV and export learner directories.
- **Actor:** L&D Administrator
- **Preconditions:** Valid Excel file uploaded.
- **Main Flow:** Admin uploads Excel file → pandas parses rows → validates Global IDs → creates/updates [Learner](#) records.
- **Implementation Status:** Fully Implemented

FR-018: Manager Team Roster View

- **Description:** Managers view progress of direct reports.
- **Actor:** Manager Learner

- **Preconditions:** Learner has subordinates (`Learner.manager_id` points to learner ID).
- **Main Flow:** Manager accesses "My Team" tab in portal → system queries `Learner.query.filter_by(manager_id=...)` → renders team completion statistics.
- **Implementation Status:** Fully Implemented

FR-019: Decoupled Backblaze B2 / MinIO Storage Integration

- **Description:** Storage service delegates file storage to S3 API endpoints when configured.
- **Actor:** System
- **Preconditions:** `STORAGE_PROVIDER=s3` set in environment.
- **Main Flow:** Upload requested → `b2_service.py` uploads byte stream via `boto3` to S3 bucket → generates direct/presigned S3 URL.
- **Implementation Status:** Fully Implemented

FR-020: Super Admin Backup, Restore & Reset

- **Description:** Super admin exports database backup, restores DB, or resets data cleanly.
 - **Actor:** Super Administrator
 - **Preconditions:** Logged in as super admin.
 - **Main Flow:** Super admin selects Backup Database → system copies `lms.db` to downloadable archive; select Reset → system re-executes `init_db_and_seed`.
 - **Implementation Status:** Fully Implemented
-

5. Non-Functional Requirements

5.1 Performance

- **Database Indexing:** Explicit database indexes on `Learner.global_id`, `Course.course_id`, `LiveClass.class_id`, `Certificate.certificate_id`, `LearnerEnrollment.learner_id`, `LearnerEnrollment.course_id`.
- **SQLite Concurrency:** Configured WAL mode (`PRAGMA journal_mode=WAL`) and normal synchronous mode (`PRAGMA synchronous=NORMAL`) to support concurrent reader threads.
- **File Payload Limit:** Maximum HTTP payload upload size capped at 1 GB (`MAX_CONTENT_LENGTH = 1024 1024 1024`).

5.2 Security

- **Admin Password Protection:** Admin passwords hashed using PBKDF2/SHA256 via Werkzeug (`generate_password_hash`).
- **CSRF Protection:** Universal CSRF protection enabled via Flask-WTF (`CSRFProtect`), with explicit exemption for B2 storage routes.
- **Security Vulnerabilities Identified:** Learner authentication lacks password verification (passwordless Global ID). Hardcoded default admin credentials in seed script.

5.3 Reliability & Availability

- **WSGI Production Support:** Production deployment configured with Waitress WSGI server (`waitress.serve`) on Windows/Linux and Gunicorn on Render (`Procfile`).

- **Fallback Storage:** Automatic local storage fallback ([uploads/](#)) if S3/MinIO cloud storage credentials are not provided.

5.4 Maintainability & Extensibility

- **Modular Blueprints:** Clean separation of concerns across 13 Flask Blueprints.
- **Alembic Migrations:** Relational database migration support enabled via Flask-Migrate.

5.5 Usability

- **Dynamic Styling & Themes:** Theme engine supporting custom skins ([navy](#), etc.) persisted per learner session.
- **Responsive Layout:** Bootstrap 5 responsive layout compatible with mobile browsers for QR attendance scanning.

<!-- Feature_Catalog.md -->

Learning Hub V3 — Detailed Feature Catalog

This document lists every implemented feature module in **Learning Hub V3**, detailing its purpose, entry point, user roles, API/backend implementation, database interactions, and implementation status.

Feature Index Matrix

Feature ID	Feature Name	Entry Point	User Role	Status
FEAT-01	Admin Authentication & Dashboard	/login , /dashboard	Admin	Fully Implemented
FEAT-02	Learner Global ID Login	/learner/login	Learner	Fully Implemented
FEAT-03	Course Management & Catalog	/courses/	Admin	Fully Implemented
FEAT-04	Multi-Format Courseware Engine	/courses/<id>/lessons	Admin / Learner	Fully Implemented
FEAT-05	SCORM 1.2 / 2004 Package Player	/courses/courseware/<id>/scorm_player	Learner	Fully Implemented
FEAT-06	Interactive Rise 360 Authoring	/courses/rise_editor/<id>	Admin	Fully Implemented
FEAT-07	Live Class Roster & Scheduling	/classes/	Admin / Facilitator	Fully Implemented
FEAT-08	Real-Time QR Attendance Engine	/attendance/	Facilitator / Learner	Fully Implemented
FEAT-09	Manual Attendance & Audit Log	/attendance/manual	Admin	Fully Implemented

FEAT-10	Assessment & Quiz Engine	<a href="/courses/<id>/assessment">/courses/<id>/assessment	Learner	Fully Implemented
FEAT-11	PDF Certificate Generation	/certificates/	Learner / Admin	Fully Implemented
FEAT-12	Certificate Verification Portal	/certificates/verify	Public	Fully Implemented
FEAT-13	Feedback Survey Repository	/feedback/	Admin / Learner	Fully Implemented
FEAT-14	Gamification (Points/Streaks/Badges)	/learners/portal	Learner	Fully Implemented
FEAT-15	Social Learning Wall	/learning_wall/	All	Fully Implemented
FEAT-16	Learner Directory & Excel Import	/learners/	Admin	Fully Implemented
FEAT-17	Manager Team Dashboard	/learners/portal	Manager Learner	Fully Implemented
FEAT-18	Support Ticket Helpdesk	/learners/issues	Learner / Admin	Fully Implemented
FEAT-19	Decoupled S3/B2 Storage	/b2_routes	System	Fully Implemented
FEAT-20	Super Admin Management	/super_admin/	Super Admin	Fully Implemented

Detailed Feature Descriptions

FEAT-01: Admin Authentication & Dashboard

- **Purpose:** Authenticates L&D administrators and presents system-wide metrics (total learners, active courses, scheduled live classes, open support tickets, completion trends).
- **User Role:** Admin / Super Admin
- **Entry Point:** <http://localhost:5000/login> → <http://localhost:5000/dashboard>
- **APIs Used:** [GET /login](#), [POST /login](#), [GET /dashboard](#), [GET /logout](#)
- **Database Operations:** Reads [AdminUser](#), [Learner](#), [Course](#), [LiveClass](#), [LmsIssue](#).
- **Status:** Fully Implemented

FEAT-02: Learner Global ID Login

- **Purpose:** Authenticates learners via passwordless Global ID lookup, calculates login streak, awards daily gamification points, and grants badges.
- **User Role:** Learner
- **Entry Point:** <http://localhost:5000/learner/login>
- **APIs Used:** [GET /learner/login](#), [POST /learner/login](#)

- **Database Operations:** Queries `Learner` by `global_id`; updates `points`, `current_streak`, `last_active_date`; inserts `LearnerBadge`.
- **Status:** Fully Implemented

FEAT-03: Course Management & Catalog

- **Purpose:** Provides full CRUD management of Self-Paced, Live Online, and Live In-Person courses with auto-generated course IDs (`CRS-SP-XXX`, `CRS-ON-XXX`, `CRS-IP-XXX`).
- **User Role:** Admin
- **Entry Point:** `http://localhost:5000/courses/`
- **APIs Used:** `GET /courses/`, `GET /courses/create`, `POST /courses/create`, `POST /courses/<id>/edit`, `POST /courses/<id>/archive`
- **Database Operations:** `Course.query`, `Course.generate_course_id()`.
- **Status:** Fully Implemented

FEAT-04: Multi-Format Courseware Engine

- **Purpose:** Supports video links (YouTube embed), uploaded PDFs, PPTX slides, text lessons, Google Drive links, and non-downloadable materials.
- **User Role:** Admin / Learner
- **Entry Point:** `http://localhost:5000/courses/<id>/lessons`
- **APIs Used:** `POST /courses/<id>/lessons/create`, `GET /courses/courseware/<id>/raw`
- **Database Operations:** `CourseLesson`, `LessonCourseware`, `CourseMaterial`.
- **Status:** Fully Implemented

FEAT-05: SCORM 1.2 / 2004 Package Player

- **Purpose:** Unzips SCORM `.zip` packages, parses `imsmanifest.xml`, and executes interactive HTML5 courseware inside an embedded iframe.
- **User Role:** Learner
- **Entry Point:** `http://localhost:5000/courses/courseware/<id>/scorm_player`
- **APIs Used:** `GET /courses/courseware/<id>/scorm_player`, `GET /courses/scorm_content/<id>/<path>`
- **Database Operations:** Reads `LessonCourseware` (type SCORM). `scorm_service.py` handles unzipping.
- **Status:** Fully Implemented

FEAT-06: Interactive Rise 360 Block Authoring

- **Purpose:** Provides a block-based courseware builder to author interactive accordions, flipcards, process diagrams, and quizzes with version control.
- **User Role:** Admin
- **Entry Point:** `http://localhost:5000/courses/rise_editor/<courseware_id>`
- **APIs Used:** `GET /courses/rise_editor/<id>`, `POST /courses/rise_editor/<id>/save`
- **Database Operations:** Creates `RiseCoursewareVersion` records storing JSON payloads.
- **Status:** Fully Implemented

FEAT-07: Live Class Roster & Scheduling

- **Purpose:** Schedules live classes (In Person or Online via Google Meet), assigns facilitators, and manages learner enrollment rosters.
- **User Role:** Admin / Facilitator
- **Entry Point:** <http://localhost:5000/classes/>
- **APIs Used:** [GET /classes/](#), [POST /classes/create](#), [POST /classes/<id>/enroll](#)
- **Database Operations:** [LiveClass](#), [LearnerEnrollment](#).
- **Status:** Fully Implemented

FEAT-08: Real-Time QR Attendance Engine

- **Purpose:** Generates dynamic QR codes for facilitators and provides mobile camera scanning for learners to mark instantaneous attendance.
- **User Role:** Facilitator / Learner
- **Entry Point:** http://localhost:5000/attendance/qr_view/<class_id>
- **APIs Used:** [GET /attendance/qr_view/<id>](#), [GET /attendance/scan/<id>](#)
- **Database Operations:** [Attendance.query.filter_by\(class_id=..., learner_id=...\)](#), creates [Attendance\(recorded_via='QR'\)](#).
- **Status:** Fully Implemented

FEAT-09: Manual Attendance & Audit Log

- **Purpose:** Allows admins to manually update learner attendance status while requiring a mandatory justification reason logged to the audit table.
- **User Role:** Admin
- **Entry Point:** http://localhost:5000/attendance/manual/<class_id>
- **APIs Used:** [POST /attendance/manual/<class_id>](#)
- **Database Operations:** Updates [Attendance](#), inserts [AuditLog\(action='MANUAL_ATTENDANCE'\)](#).
- **Status:** Fully Implemented

FEAT-10: Assessment & Quiz Engine

- **Purpose:** Conducts MCQ assessments (Pre-Course, Lesson Post, End-Course, Standalone Quiz), evaluates score percentages, and determines passing criteria (default 80%).
- **User Role:** Learner
- **Entry Point:** <http://localhost:5000/courses/<id>/assessment/<type>>
- **APIs Used:** [GET /courses/<id>/assessment/<type>](#), [POST /courses/<id>/assessment/<type>/submit](#)
- **Database Operations:** Inserts [AssessmentAttempt](#), updates [LearnerEnrollment.completion_status](#).
- **Status:** Fully Implemented

FEAT-11: PDF Certificate Generation

- **Purpose:** Automatically generates verifiable, downloadable PDF certificates via ReportLab when course completion requirements are satisfied.
- **User Role:** Learner / Admin
- **Entry Point:** http://localhost:5000/certificates/download/<cert_id>
- **APIs Used:** [GET /certificates/download/<id>](#), [GET /certificates/generate/<enrollment_id>](#)
- **Database Operations:** Creates [Certificate](#) with unique [CERT-XXXXXX](#) ID. [pdf_service.py](#) outputs PDF.
- **Status:** Fully Implemented

FEAT-12: Certificate Verification Portal

- **Purpose:** Provides a public webpage to verify the authenticity of issued certificates using their unique ID.
- **User Role:** Public
- **Entry Point:** <http://localhost:5000/certificates/verify>
- **APIs Used:** [GET /certificates/verify?id=CERT-XXXXXX](#)
- **Database Operations:** Queries [Certificate](#) by [certificate_id](#).
- **Status:** Fully Implemented

FEAT-13: Feedback Survey Repository

- **Purpose:** Manages reusable feedback question repositories (MCQ & Text) and collects survey responses after course/class completion.
- **User Role:** Admin / Learner
- **Entry Point:** <http://localhost:5000/feedback/>
- **APIs Used:** [GET /feedback/](#), [POST /feedback/create](#), [POST /feedback/submit/<repo_id>](#)
- **Database Operations:** [FeedbackRepository](#), [FeedbackQuestion](#), [FeedbackResponse](#).
- **Status:** Fully Implemented

FEAT-14: Gamification Engine

- **Purpose:** Tracks daily login streaks, awards experience points, and automatically bestows badges ([Streak Master](#) ■, [Fast Learner](#)).
- **User Role:** Learner
- **Entry Point:** <http://localhost:5000/learners/portal>
- **APIs Used:** Integrated into auth and course completion hooks ([gamification.py](#)).
- **Database Operations:** [Learner.points](#), [Learner.current_streak](#), [LearnerBadge](#).
- **Status:** Fully Implemented

FEAT-15: Social Learning Wall

- **Purpose:** Community wall displaying system updates, birthday wishes, course completions, earned certificates, reactions (like, love, celebrate, clap, fire), and comments.
- **User Role:** Learner / Admin

- **Entry Point:** http://localhost:5000/learning_wall/
- **APIs Used:** [GET /learning_wall/](#), [POST /learning_wall/post](#), [POST /learning_wall/react](#), [POST /learning_wall/comment](#)
- **Database Operations:** [LearningWallPost](#), [LearningWallReaction](#), [LearningWallComment](#).
- **Status:** Fully Implemented

FEAT-16: Learner Directory & Bulk Import/Export

- **Purpose:** Admin directory listing all learners, filtering by department/location/branch, Excel bulk import via pandas, and Excel export.
- **User Role:** Admin
- **Entry Point:** <http://localhost:5000/learners/>
- **APIs Used:** [GET /learners/](#), [POST /learners/import](#), [GET /learners/export](#)
- **Database Operations:** [Learner.query](#), bulk insert/update.
- **Status:** Fully Implemented

FEAT-17: Manager Team Dashboard

- **Purpose:** Allows learners with direct reports ([manager_id](#)) to monitor team members' course progress, completion status, and extension requests.
- **User Role:** Manager Learner
- **Entry Point:** <http://localhost:5000/learners/portal> (My Team tab)
- **APIs Used:** [GET /learners/portal](#)
- **Database Operations:** [Learner.subordinates](#), [LearnerEnrollment.query](#).
- **Status:** Fully Implemented

FEAT-18: Support Ticket Helpdesk

- **Purpose:** Allows learners to file technical/content/certificate support tickets and enables admins to resolve them.
- **User Role:** Learner / Admin
- **Entry Point:** <http://localhost:5000/learners/issues>
- **APIs Used:** [POST /learners/issues/create](#), [POST /learners/issues/<id>/resolve](#)
- **Database Operations:** [LmsIssue](#).
- **Status:** Fully Implemented

FEAT-19: Decoupled S3/Backblaze B2 Storage

- **Purpose:** Decouples media file storage from local server to S3-compatible cloud storage (Backblaze B2 or MinIO) using [boto3](#).
- **User Role:** System
- **Entry Point:** [/b2_routes](#)
- **APIs Used:** [POST /b2/upload](#), [GET /b2/file/<filename>](#)
- **Database Operations:** Stores S3 file keys in database models.
- **Status:** Fully Implemented

FEAT-20: Super Admin Management Console

- **Purpose:** Provides system-level database backup download, DB restore, system reset, and audit log inspection.
 - **User Role:** Super Admin
 - **Entry Point:** http://localhost:5000/super_admin/
 - **APIs Used:** [GET /super_admin/](#), [GET /super_admin/backup_db](#), [POST /super_admin/reset_data](#)
 - **Database Operations:** [AuditLog.query](#), SQLite file copy.
 - **Status:** Fully Implemented
-

<!-- User_Roles.md -->

Learning Hub V3 — User Roles & Permissions

This document details the user roles, access control mechanisms, permission scopes, accessible modules, and restrictions in **Learning Hub V3**.

1. Role Hierarchy

```
graph TD
  SA[Super Administrator] --> Admin[L&D Administrator]
  Admin --> Fac[Session Facilitator]
  Admin --> Mgr[Manager / Department Head]
  Mgr --> Learner[Learner / Employee]
  Fac --> Learner
```

2. Detailed Role Definitions

2.1 Super Administrator ([admin](#))

- **Description:** Top-level system operations administrator responsible for platform infrastructure, storage provider settings, audit compliance, and database maintenance.
- **Authentication:** Form-based authentication ([/login](#)) with bcrypt/pbkdf2 password hashing.
- **Accessible Modules:**
 - [/super_admin/*](#) — System backup, database reset, storage provider settings, audit log viewer
 - [/dashboard](#) — Platform overview analytics
 - All L&D Administrator modules
- **Permissions:**
 - Download SQLite database backups
 - Execute full database reset and re-seeding

- Inspect system-wide audit logs ([AuditLog](#))
- Configure Backblaze B2 / MinIO S3 credentials
- **Restrictions:** None.

2.2 L&D Administrator

- **Description:** Operational manager responsible for curriculum design, courseware authoring, live class scheduling, learner enrollment, attendance override, feedback creation, and compliance report export.
- **Authentication:** Form-based authentication ([/login](#)).
- **Accessible Modules:**
 - [/courses/](#)* — Course CRUD, lesson creation, Rise 360 authoring, SCORM upload
 - [/classes/](#)* — Live class scheduling, facilitator assignment, roster lock/unlock
 - [/learners/](#)* — Learner directory, Excel import/export, issue ticket management
 - [/attendance/](#)* — Manual attendance override, attendance reports
 - [/feedback/](#)* — Survey questionnaire builder, feedback analytics
 - [/certificates/](#)* — Certificate re-issuance, cert management
 - [/reports/](#)* — Excel/PDF report generation
 - [/quizzes/](#)* — Standalone quiz builder
- **Permissions:**
 - Create, update, archive, delete courses and live classes
 - Author Rise 360 interactive block content
 - Manually override attendance with mandatory reason logging
 - Resolve support tickets
- **Restrictions:** Cannot perform system-level database wipes or export DB backups unless logged into Super Admin console.

2.3 Session Facilitator

- **Description:** Instructor or subject matter expert assigned to conduct live online or in-person sessions.
- **Authentication:** Passwordless Global ID login ([/learner/login](#)).
- **Accessible Modules:**
 - [/classes/live/<class_id>](#) — Live session roster view
 - [/attendance/qr_view/<class_id>](#) — Facilitator QR code display screen
 - Standard Learner Portal modules
- **Permissions:**
 - Display live class QR code for learner attendance scanning
 - View live class roster and attendance status
- **Restrictions:** Cannot create or archive courses or modify global platform settings.

2.4 Manager / Department Head

- **Description:** Organizational manager overseeing direct report learners.

- **Authentication:** Passwordless Global ID login (</learner/login>).
- **Accessible Modules:**
 - </learners/portal> — Learner Portal with "My Team" tab
 - Standard Learner Portal modules
- **Permissions:**
 - View progress, course completion rates, and assessment scores of direct reports (`Learner.manager_id == current_user.id`)
 - View and review deadline extension requests from direct reports
- **Restrictions:** Cannot view learners outside their direct reporting hierarchy.

2.5 Learner / Employee

- **Description:** End-user taking self-paced or live courses.
- **Authentication:** Passwordless Global ID login (</learner/login>).
- **Accessible Modules:**
 - </learners/portal> — Dashboard, active courses, badges, points, streak tracker
 - </courses/<id>/flow> — Video playback, SCORM execution, PPT slide viewer
 - /attendance/scan/<class_id> — Mobile QR attendance camera scanner
 - /learning_wall/* — Social wall posts, reactions, comments
 - /certificates/* — Earned certificate PDF download
 - </learners/issues> — Log support ticket
- **Permissions:**
 - Complete enrolled courses and assessments
 - Scan QR code to mark attendance
 - Earn gamification points, streaks, and badges
 - Download earned PDF certificates
 - Post comments and reactions on the Learning Wall
- **Restrictions:** Cannot access administrative views, edit courses, or modify attendance records.

<!-- User_Flows.md -->

Learning Hub V3 — User Workflows & Flow Diagrams

This document details the step-by-step workflows for major user journeys in **Learning Hub V3**, including happy paths, alternative paths, and error states.

1. Learner Authentication & Portal Entry

```

flowchart TD
    Start([Learner Accesses /learner/login]) --> EnterID[Enter Global ID e.g. 10001]
    EnterID --> Submit[Submit Form]
    Submit --> ValidID[Valid ID]
    ValidID --> FindLearner[Find Learner in DB]
    Submit --> InvalidID[Invalid ID]
    InvalidID --> ErrMsg[Display Error: Learner Not Found]
    ErrMsg --> EnterID
    FindLearner --> StreakCheck[Check Last Active Date]
    StreakCheck --> LoggedIn[Logged in today]
    LoggedIn --> MaintainStreak[Keep Current Streak]
    StreakCheck --> Yesterday[Yesterday]
    Yesterday --> IncStreak[Increment Streak + Award Points]
    StreakCheck --> Older[Older]
    Older --> ResetStreak[Reset Streak to 1 + Award Login Points]
    IncStreak --> CheckBadge[Streak >= 5?]
    CheckBadge --> Yes[Yes]
    Yes --> AwardBadge[Award Streak Master Badge]
    CheckBadge --> No[No]
    No --> Portal[Redirect to /learners/portal]
    AwardBadge --> Portal
    MaintainStreak --> Portal
    ResetStreak --> Portal
  
```

2. Self-Paced Course Completion & Certification Flow

```

flowchart TD
    Portal([Learner Portal]) --> SelectCourse[Select Self-Paced Course]
    SelectCourse --> CheckPreAss[Pre-Assessment Exists?]
    CheckPreAss --> Yes1[Yes]
    Yes1 --> TakePre[Take Pre-Course Assessment]
    CheckPreAss --> No1[No]
    No1 --> WatchLessons1[View Lesson Content / Video / SCORM]
    TakePre --> WatchLessons1
    WatchLessons1 --> CheckMinTime[Spent Min Time on Lesson?]
    CheckMinTime --> No2[No]
    No2 --> WaitTime[Display Time Remaining Banner]
    WaitTime --> WatchLessons1
    CheckMinTime --> Yes2[Yes]
    Yes2 --> LessonAss[Lesson Assessment Exists?]
    LessonAss --> Yes3[Yes]
    Yes3 --> TakeLessonAss[Submit MCQ Answers]
    LessonAss --> No2
    No2 --> NextLesson[More Lessons?]
    TakeLessonAss --> Score[Score < 80%]
    Score --> RetryAss[Retry Assessment max 3 attempts]
    RetryAss --> TakeLessonAss
    TakeLessonAss --> Score2[Score >= 80%]
    Score2 --> NextLesson2[Next Lesson]
    NextLesson2 --> Yes4[Yes]
    Yes4 --> WatchLessons2[Watch Lessons]
    NextLesson2 --> No3[No]
    No3 --> TakePost[Take Final Post-Course Assessment]
    TakePost --> CheckScore[Score >= Pass Percentage?]
    CheckScore --> No4[No]
    No4 --> Fail[Mark Status: Failed / Retry]
    CheckScore --> Yes5[Yes]
    Yes5 --> FeedbackSurvey[Redirect to Feedback Survey]
    FeedbackSurvey --> SubmitFeedback[Submit Survey Responses]
    SubmitFeedback --> Complete[Mark Status: Completed & Award Points]
    Complete --> GenCert[Generate ReportLab PDF Certificate]
    GenCert --> DownloadCert[Download CERT-XXXXX.pdf]
  
```

3. Live Class QR Attendance & Audit Override Flow

```

flowchart TD
    Facilitator([Facilitator opens /attendance/qr_view/CLASS_ID]) --> DisplayQR[Display QR Code on Screen]
    DisplayQR --> LearnerScan[Learner scans QR code via Mobile Browser]
    LearnerScan --> CheckAuth[Learner Logged In?]
    CheckAuth --> No1[No]
    No1 --> RedirectLogin[Redirect to Learner Login with classId param]
    RedirectLogin --> LearnerScan
    CheckAuth --> Yes1[Yes]
    Yes1 --> VerifyRoster[Learner in Class Roster?]
    VerifyRoster --> No2[No]
    No2 --> Denied[Display Error: Not Enrolled in Class]
    VerifyRoster --> Yes2[Yes]
    Yes2 --> MarkAttendance[Record Attendance: Present, QR]
    MarkAttendance --> Success([Display Green Confirmation Screen])

    subgraph AdminManualOverride[Audit]
        Admin([Admin opens /attendance/manual/CLASS_ID]) --> UpdateStatus[Select Learner & Change Status]
        UpdateStatus --> EnterReason[Enter Mandatory Reason]
        EnterReason --> SaveAudit[Save Attendance & Write AuditLog]
    end
  
```

```
end
```

4. Support Ticket Resolution Flow

```
flowchart TD
  Learner[Learner in Portal] --> ClickHelp[Click Support / Report Issue]
  ClickHelp --> FillForm[Select Category: Technical/Content/Certificate & Enter Description]
  FillForm --> SubmitIssue[Create LmsIssue Record status=Open]
  SubmitIssue --> AdminView[Admin views /learners/issues]
  AdminView --> ReviewTicket[Review Issue Details]
  ReviewTicket --> ClickResolve[Click Resolve Ticket]
  ClickResolve --> UpdateDB[Update LmsIssue status=Resolved & set resolved_at]
  UpdateDB --> NotifLearner([Learner sees ticket marked Resolved])
```

```
<!-- System_Architecture.md -->
```

Learning Hub V3 — System Architecture Document

This document provides the high-level technical architecture of **Learning Hub V3**, detailing the interaction between clients, server application layers, databases, storage engines, and external integrations.

1. High-Level System Architecture Diagram

```
flowchart TD
  subgraph Clients["Clients / User Interfaces"]
    Browser[Desktop Browser - Admin / Learner]
    Mobile[Mobile Browser - Learner QR Scanner]
    Public[External Verifier - Public Portal]
  end

  subgraph CDN["Edge & Security Layer"]
    Cloudflare[Cloudflare CDN & DDoS Defense - DNS Proxy]
  end

  subgraph ApplicationServer["Application Server Layer (Flask Monolith)"]
    WSGI[Waitress WSGI / Gunicorn Server]

    subgraph Blueprints["Flask Blueprints Routing Layer"]
      AuthBP[auth_bp / login]
      DashBP[dashboard_bp]
      CourseBP[courses_bp]
      ClassBP[classes_bp]
      LearnerBP[learners_bp]
      AttBP[attendance_bp]
      CertBP[certificates_bp]
      WallBP[learning_wall_bp]
      SuperAdminBP[super_admin_bp]
      B2BP[b2_bp]
    end

    subgraph Services["Core Application Services"]
      SCORM[scorm_service.py - Manifest & ZIP Parser]
      PDF[pdf_service.py - ReportLab PDF Engine]
      QR[qr_service.py - QRCode Generator]
    end
  end
```

```

B2[b2_service.py - S3/B2 Upload Manager]
GDrive[gdrive_service.py - Google Drive Embed Parser]
Gamify[gamification.py - Streak & Points Engine]
end
end

subgraph DataStore["Persistence & Storage Layer"]
DB[(SQLAlchemy ORM: SQLite WAL / PostgreSQL)]
LocalStorage[(Local File Storage: uploads/ & qr_codes/)]
CloudStorage[(Decoupled S3 / Backblaze B2 / MinIO Bucket)]
end

subgraph External["External Integrations"]
YouTube[YouTube Video Embeds]
GoogleMeet[Google Meet Live Virtual Links]
end

Browser --> Cloudflare
Mobile --> Cloudflare
Public --> Cloudflare
Cloudflare --> WSGI
WSGI --> Blueprints
Blueprints --> Services
Services --> DB
Services --> LocalStorage
Services --> CloudStorage
Services --> YouTube
Services --> GoogleMeet

```

2. Architecture Layer Breakdown

2.1 Edge & Security Layer

- **Cloudflare CDN (Optional Production Deployment):** Proxies inbound HTTPS traffic to hide application server IP, caches static assets ([/static/](#)), and defends against distributed denial-of-service (DDoS) attacks.

2.2 Application Server Layer

- **WSGI Runner:** Runs on Waitress (Production Windows/Linux) or Gunicorn (Linux/Render) listening on configurable port (default 5000).
- **Flask Framework (3.1.3):** Handles request lifecycle, session management, CSRF validation, and template rendering.
- **13 Modular Blueprints:** Organizes routing by feature module.

2.3 Core Services Subsystem

- **SCORM Engine ([scorm_service.py](#)):** Handles unzipping SCORM zip packages, extracting [imsmanifest.xml](#), resolving root launch HTML files, and serving isolated asset streams.
- **PDF Engine ([pdf_service.py](#)):** Uses [reportlab](#) to programmatically build vector PDF certificates with custom typography, borders, logos, and QR codes.
- **QR Engine ([qr_service.py](#)):** Generates PNG QR code images encoding URL routes for instantaneous mobile attendance scanning.
- **Storage Subsystem ([b2_service.py](#), [storage_service.py](#)):** Dual-mode storage engine. Automatically toggles between local filesystem ([uploads/](#)) and S3-compatible cloud storage (Backblaze B2 / MinIO) using [boto3](#).

2.4 Persistence Subsystem

- **SQLAlchemy 2.0 ORM:** Abstracts SQL database queries.
- **Database Backends:**
- **SQLite** (`lms.db`): Pre-configured for local dev and lightweight deployment with `PRAGMA journal_mode=WAL` and `PRAGMA synchronous=NORMAL`.
- **PostgreSQL** (`DATABASE_URL`): Configured via environment variable for production scale (60,000+ active learners).

3. Data Flow Architecture

3.1 Courseware Playback Data Flow

```
Learner Request → Course BP → Fetch CourseLesson & LessonCourseware → Check Type:
■■■ Video URL → Render HTML5 iframe (YouTube / direct stream)
■■■ PDF / PPT → Parse via pypdfium2 / pptx_parser → Render embedded canvas / slides
■■■ SCORM → Extract ZIP via scorm_service → Launch index.html inside iframe
```

3.2 Certificate Issuance Data Flow

```
Course Completed & Feedback Submitted → Trigger generate_certificate() →
■■■ Generate Unique CERT-UUID ID
■■■ Invoke pdf_service.py ReportLab canvas
■■■ Draw text, borders, verification URL, QR code
■■■ Output PDF file to uploads/certificates/
■■■ Save Certificate record in Database
```

<!-- Frontend_Architecture.md -->

Learning Hub V3 — Frontend Architecture Document

This document describes the frontend design, layout inheritance, component structure, asset management, and browser interactions in **Learning Hub V3**.

1. Overview

The frontend of **Learning Hub V3** is constructed using server-side rendered HTML5 templates (Flask Jinja2) augmented with modular CSS stylesheets, Bootstrap 5 UI framework, FontAwesome 6 icons, Chart.js for data visualization, and client-side JavaScript utilities.

2. Template Structure & Layout Hierarchy

```
app/templates/
■■■ base.html # Root master layout (Navbar, Sidebar, Flash Banners, Modals, Footer)
■■■ auth/ # Admin & Learner login pages
■ ■■■ admin_login.html
■ ■■■ learner_login.html
```

```
■■■ dashboard/ # Main administrative executive dashboard
■ ■■■ index.html
■■■ learner_portal/ # Learner Portal layout and tabbed views
■ ■■■ portal.html
■■■ courses/ # Course creation, list, detail, lesson editor, Rise builder
■ ■■■ index.html
■ ■■■ create.html
■ ■■■ detail.html
■ ■■■ rise_editor.html
■ ■■■ scorm_player.html
■■■ classes/ # Live class schedule, creation, roster management
■ ■■■ index.html
■ ■■■ detail.html
■■■ learners/ # Learner directory, profile, issue desk
■ ■■■ index.html
■ ■■■ profile.html
■ ■■■ issues.html
■■■ attendance/ # QR view, camera scanner, manual attendance table
■ ■■■ qr_view.html
■ ■■■ scan.html
■ ■■■ manual.html
■■■ feedback/ # Feedback repository builder and survey form
■ ■■■ index.html
■ ■■■ survey.html
■■■ certificates/ # Certificate viewer & public verification portal
■ ■■■ view.html
■ ■■■ verify.html
■■■ learning_wall/ # Community social wall feed & comment modal
■ ■■■ index.html
■■■ super_admin/ # Super admin system operations console
■ ■■■ index.html
■■■ errors/ # Custom HTTP 404, 500, 413 error templates
■■■ 404.html
■■■ 500.html
```

3. Component Architecture & Master Layout (base.html)

The master layout `base.html` provides standard shell elements:

- 1. Global Header / Top Navigation:** Displays branding logo, global Search bar, Notification Bell dropdown (with real-time unread counter), Gamification Points pill, Learner Profile picture/avatar, and Theme Switcher.
- 2. Sidebar Navigation:** Role-aware dynamic menu rendering distinct navigation links for Admin (`/dashboard`, `/courses`, `/classes`, `/learners`, `/attendance`, `/feedback`, `/certificates`, `/reports`, `/quizzes`, `/learning_wall`, `/super_admin`) and Learner (`/learners/portal`, `/learning_wall`).
- 3. Flash Message Toast Banners:** Rendered dynamically from Flask's `get_flashed_messages()` with alert levels (`success`, `danger`, `warning`, `info`).
- 4. Modal Containers:** Reusable HTML modals for profile picture upload, issue ticketing, feedback survey popups, and confirmation dialogs.

4. Theme System & Dynamic Styling

The UI supports a dynamic theme engine initialized via global Jinja context processors (`inject_global_vars()` in `app/__init__.py`):

- Default theme variant: `navy` (deep dark blue navbar headers, sleek card borders, glassmorphic accents).

- Theme preferences are saved per learner in `Learner.theme` and synced to `session['learner_theme']`.
- Custom CSS utility classes format badges (`bg-teal-subtle text-teal`), progress bars, and stats widgets.

5. Key Client-Side JavaScript Libraries

Library	Version / Source	Purpose in Application
Bootstrap 5	CDN JS / CSS bundle	Grid layout, responsive modals, collapse toggles, dropdowns
FontAwesome 6	CDN icon fonts	Comprehensive UI icons for badges, course modes, and menu items
Chart.js	CDN script	Renders interactive analytics charts on /dashboard and /reports
Html5-QRCode Scanner	CDN JS plugin	Camera access utility for reading attendance QR codes on /attendance/scan
QRCode.js	CDN script	Client-side QR rendering backup for live class session codes

<!-- Backend_Architecture.md -->

Learning Hub V3 — Backend Architecture Document

This document describes the backend design, application factory, blueprint routing structure, service modules, middleware, error handling, and authorization in **Learning Hub V3**.

1. Overview

The backend of **Learning Hub V3** is constructed as a modular Python Flask monolith using Flask Blueprints, SQLAlchemy ORM, Flask-WTF CSRF protection, Flask-Migrate database migrations, and specialized service modules.

2. Application Factory (create_app)

The application entry point (`run.py` and `api/index.py`) invokes `create_app()` defined in `app/__init__.py`:

```
def create_app(config_class=Config):
    app = Flask(__name__)
    app.config.from_object(config_class)

    # Initialize extensions
    config_class.init_app(app)
```

```

db.init_app(app)
csrf.init_app(app)
migrate.init_app(app, db)

# Configure SQLite PRAGMAS for high concurrency
# Register 13 Blueprints
# Register custom Jinja filters & global context processors
# Register custom HTTP error handlers (404, 500, 413)

```

3. Blueprint Roster & Routing Architecture

Blueprint Name	Prefix	Python Source File	Purpose
auth_bp	/	app/routes/auth.py	Admin login/logout, Learner Global ID login
dashboard_bp	/dashboard	app/routes/dashboard.py	Executive analytics dashboard
courses_bp	/courses	app/routes/courses.py	Course CRUD, lesson courseware, Rise editor, SCORM
classes_bp	/classes	app/routes/classes.py	Live class scheduling, rosters, seating
learners_bp	/learners	app/routes/learners.py	Learner directory, profile, Excel import/export, issues
attendance_bp	/attendance	app/routes/attendance.py	QR view, mobile scan, manual override & audit
feedback_bp	/feedback	app/routes/feedback.py	Feedback repositories, question builder, survey submit
certificates_bp	/certificates	app/routes/certificates.py	Certificate download, PDF generation, verification
reports_bp	/reports	app/routes/reports.py	Compliance reporting & Excel/PDF export
learning_wall_bp	/learning_wall	app/routes/learning_wall.py	Social posts, reactions, comments
super_admin_bp	/super_admin	app/routes/super_admin.py	System backup, restore, reset, audit trail
quizzes_bp	/quizzes	app/routes/quizzes.py	Standalone quiz creation & management
b2_bp	/b2	app/routes/b2_routes.py	Backblaze B2 S3 API upload & file retrieval

4. Business Logic Service Layer

```

app/services/
■■■ assessment_service.py # Evaluation of MCQ assessments, score calculations, pass thresholds

```

```
■■■ b2_service.py # Backblaze B2 / S3 client wrapper using boto3
■■■ gdrive_service.py # Google Drive URL parser (extracts File IDs, transforms to embed URLs)
■■■ learning_wall_service.py # Automated post generation (birthdays, course completion announcements)
■■■ lock_service.py # Live class locking / unlocking rules & constraint checking
■■■ pdf_service.py # ReportLab PDF canvas drawing engine for certificates
■■■ qr_service.py # qrcode library wrapper for generating PNG QR streams
■■■ report_service.py # Aggregates completion data and formats pandas / openpyxl DataFrames
■■■ scorm_service.py # ZipFile extraction of SCORM archives and manifest parsing
■■■ storage_service.py # Abstract storage manager (local filesystem vs S3 cloud)
```

5. Middleware & Security Controls

- 1. **CSRF Protection:** Handled globally by `CSRFProtect(app)`. Blueprint `b2_bp` is explicitly exempted via `csrf.exempt(b2_bp)` to allow direct REST API file payload uploads.
- 2. **Session Context Injection:** Global context processor `inject_global_vars()` runs on every template request to resolve `learner_id`, `admin_logged_in`, unread notification counts, and profile picture paths.
- 3. **Payload Threshold Enforcement:** Request payload max size configured to 1 GB (`MAX_CONTENT_LENGTH = 1024 1024 1024`). HTTP 413 error handler captures file size limit violations gracefully.

<!-- Database_Architecture.md -->

Learning Hub V3 — Database Architecture Document

This document provides a technical explanation of the relational data architecture, ORM models, primary/foreign keys, indexes, and database engine configurations in **Learning Hub V3**.

1. Overview

Learning Hub V3 uses **SQLAlchemy 2.0 ORM** to manage relational tables across 14 database models.

- **Primary Database Engine (Local Dev & Standalone):** SQLite (`lms.db`) configured with WAL mode (`PRAGMA journal_mode=WAL`) and normal synchronization (`PRAGMA synchronous=NORMAL`) for concurrent reads and writes.
- **Production Database Engine:** PostgreSQL support configured via `DATABASE_URL` environment variable with `psycopg2-binary` driver.
- **Schema Management:** Managed via Flask-Migrate (Alembic) with fallback dynamic schema modification statements in `app/seed.py`.

2. Table Summary Matrix

Table Name	Primary Key	Key Foreign Keys	Purpose
------------	-------------	------------------	---------

admin_users	id (Integer)	None	Stores L&D administrator authentication records
learners	id (Integer)	manager_id -> learners.id	Stores learner profiles, points, streaks, departments
courses	id (Integer)	feedback_repo_id , pre_quiz_id , post_quiz_id	Stores self-paced and live course metadata
course_lessons	id (Integer)	course_id , pre_quiz_id , post_quiz_id	Stores lesson modules within courses
lesson_courseware	id (Integer)	lesson_id	Stores multi-format courseware files, SCORM, video links
courseware_audio_tracks	id (Integer)	courseware_id	Stores multilingual audio tracks for courseware
course_materials	id (Integer)	course_id	Stores supplemental materials and download toggles
course_assessments	id (Integer)	course_id , lesson_id	MCQ question bank for pre, lesson post, and course end tests
live_classes	id (Integer)	course_id , facilitator_id , co_facilitator_id , quiz_id	Live in-person campus and virtual online sessions
learner_enrollments	id (Integer)	learner_id , course_id , class_id	Enrolls learners in courses/classes, tracks status & scores
assessment_attempts	id (Integer)	enrollment_id , lesson_id	Tracks individual MCQ assessment submissions & scores
lesson_reviews	id (Integer)	enrollment_id , lesson_id	Audit log of completed lessons per enrollment
attendances	id (Integer)	class_id , learner_id	Live class attendance records (QR or Manual)
certificates	id (Integer)	learner_id , course_id	Issued PDF certificate records with unique UUID IDs
external_certificates	id (Integer)	learner_id	External certificates uploaded by learners
learner_badges	id (Integer)	learner_id	Gamification badges awarded to learners
learner_notifications	id (Integer)	learner_id , course_id , lesson_id	In-app notification messages for learners
feedback_repositories	id (Integer)	None	Reusable survey questionnaire templates

<code>feedback_questions</code>	<code>id</code> (Integer)	<code>repo_id</code>	Individual survey questions (MCQ or Text)
<code>feedback_responses</code>	<code>id</code> (Integer)	<code>repo_id</code> , <code>class_id</code> , <code>learner_id</code>	Learner survey response submissions (JSON dictionary)
<code>learning_wall_posts</code>	<code>id</code> (Integer)	<code>learner_id</code> , <code>course_id</code>	Social community wall announcements and posts
<code>learning_wall_reactions</code>	<code>id</code> (Integer)	<code>post_id</code>	Reactions (<code>like</code> , <code>love</code> , <code>celebrate</code> , <code>clap</code> , <code>fire</code>)
<code>learning_wall_comments</code>	<code>id</code> (Integer)	<code>post_id</code>	Text comments on community wall posts
<code>lms_issues</code>	<code>id</code> (Integer)	<code>learner_id</code>	Support ticket helpdesk records
<code>audit_logs</code>	<code>id</code> (Integer)	None	Compliance audit trail for manual attendance & overrides
<code>quizzes</code>	<code>id</code> (Integer)	None	Standalone quiz metadata
<code>quiz_questions</code>	<code>id</code> (Integer)	<code>quiz_id</code>	Questions belonging to standalone quizzes
<code>rise_courseware_version</code>	<code>id</code> (Integer)	<code>courseware_id</code>	Versioned JSON block content for Rise courseware
<code>learner_block_progress</code>	<code>id</code> (Integer)	<code>learner_id</code> , <code>courseware_id</code>	Granular learner progress per Rise 360 interactive block

3. Explicit Database Indexes

To optimize high-concurrency lookups for 60,000+ active learners, the following columns feature explicit B-Tree database indexes:

1. `learners.global_id` (`unique=True`, `index=True`) — Instant learner lookup during login.
2. `courses.course_id` (`unique=True`, `index=True`) — Course lookup by human-readable ID (`CRS-SP-001`).
3. `live_classes.class_id` (`unique=True`, `index=True`) — Class lookup for QR scanning (`CRS-CLS-000001`).
4. `certificates.certificate_id` (`unique=True`, `index=True`) — Public verification portal lookup (`CERT-XXXXXX`).
5. `learner_enrollments.learner_id` (`index=True`) — Learner portal enrollment queries.
6. `learner_enrollments.course_id` (`index=True`) — Course enrollment roster queries.
7. `learners.manager_id` (`index=True`) — Subordinate team queries.
8. `learners.points` (`index=True`) — Leaderboard ranking queries.
9. `rise_courseware_version.courseware_id` (`index=True`) — Rise block version queries.
10. `learner_block_progress.learner_id`, `courseware_id` (`index=True`) — Block progress queries. Unique constraint on (`learner_id`, `courseware_id`, `block_id`).

<!-- API_Documentation.md -->

Learning Hub V3 — Complete API Documentation & Endpoint Inventory

This document provides complete documentation for every HTTP route and API endpoint implemented in **Learning Hub V3**.

1. API Endpoint Inventory Table

API ID	Method	Endpoint	Purpose	Auth Required	Status
API-001	GET	/	Root redirect to dashboard or login	No	Active
API-002	GET, POST	/login	L&D Admin login page and authentication	No	Active
API-003	GET	/logout	Clears session data and logs user out	Yes (Session)	Active
API-004	GET, POST	/learner/login	Passwordless Learner Global ID login	No	Active
API-005	GET	/dashboard	Renders administrative executive dashboard	Yes (Admin)	Active
API-006	GET	/courses/	Lists all courses (Self Paced, Live Online, In Person)	Yes (Admin)	Active
API-007	GET, POST	/courses/create	Renders creation form & creates new course	Yes (Admin)	Active
API-008	GET, POST	/courses/<id>/edit	Renders edit form & updates course metadata	Yes (Admin)	Active
API-009	POST	/courses/<id>/archive	Archives a course (<code>is_archived=True</code>)	Yes (Admin)	Active
API-010	GET, POST	/courses/<id>/lessons/create	Adds a new lesson to a course	Yes (Admin)	Active

API-011	GET	/courses/courseware/<id>/raw	Streams raw courseware file (PDF, MP4, SCORM)	Yes (Session)	Active
API-012	GET	/courses/courseware/<id>/scorm_player	Serves embedded SCORM player iframe	Yes (Session)	Active
API-013	GET, POST	/courses/rise_editor/<courseware_id>	Rise 360 block courseware interactive editor	Yes (Admin)	Active
API-014	GET, POST	/courses/<id>/assessment/<type>	Serves MCQ assessment & evaluates answers	Yes (Learner)	Active
API-015	GET	/classes/	Lists all scheduled live classes	Yes (Admin)	Active
API-016	GET, POST	/classes/create	Schedules a new live online / in-person class	Yes (Admin)	Active
API-017	GET, POST	/classes/<id>/enroll	Enrolls learners into live class roster	Yes (Admin)	Active
API-018	POST	/classes/<id>/lock	Locks class roster against attendance edits	Yes (Admin)	Active
API-019	POST	/classes/<id>/unlock	Unlocks class roster with mandatory audit reason	Yes (Admin)	Active
API-020	GET	/attendance/	Lists attendance records & live class QR links	Yes (Admin)	Active
API-021	GET	/attendance/qr_view/<class_id>	Facilitator view displaying live QR code	Yes (Session)	Active
API-022	GET	/attendance/scanner/<class_id>	Learner mobile camera scanner endpoint	Yes (Learner)	Active
API-023	GET, POST	/attendance/manual/<class_id>	Admin manual attendance override & audit log	Yes (Admin)	Active
API-024	GET	/learners/	Lists learner directory with filters	Yes (Admin)	Active
API-025	POST	/learners/import	Bulk imports learners from uploaded Excel file	Yes (Admin)	Active

API-026	GET	/learners/export	Exports learner roster to downloadable Excel	Yes (Admin)	Active
API-027	GET	/learners/portal	Main Learner Portal view (courses, badges, team)	Yes (Learner)	Active
API-028	GET, POST	/learners/issues	Learner support ticket log & creation	Yes (Learner)	Active
API-029	POST	/learners/issues/<id>/resolve	Admin marks support ticket resolved	Yes (Admin)	Active
API-030	GET	/feedback/	Lists feedback question repositories	Yes (Admin)	Active
API-031	GET, POST	/feedback/create	Creates new feedback question repository	Yes (Admin)	Active
API-032	GET, POST	/feedback/submit/<repo_id>	Learner submits session feedback responses	Yes (Learner)	Active
API-033	GET	/certificates/download/<cert_id>	Downloads generated PDF certificate	Yes (Learner)	Active
API-034	GET	/certificates/verify	Public certificate authenticity verification	No	Active
API-035	GET	/reports/	Executive compliance analytics & exports	Yes (Admin)	Active
API-036	GET, POST	/learning_wall/	Renders social wall, creates posts/reactions	Yes (Session)	Active
API-037	GET	/super_admin/	Renders Super Admin console	Yes (SuperAdmin)	Active
API-038	GET	/super_admin/backup_db	Downloads database backup zip/db file	Yes (SuperAdmin)	Active
API-039	POST	/super_admin/reset_data	Triggers database wipe and initial re-seeding	Yes (SuperAdmin)	Active
API-040	POST	/b2/upload	Direct file upload endpoint to S3/B2 storage	Exempt (CSRF)	Active

2. Detailed Endpoint Documentation Examples

API-002: L&D Admin Login

- **HTTP Method:** `POST`
- **Endpoint:** `/login`
- **Purpose:** Authenticates administrator credentials and initializes admin session.
- **Authentication:** None required prior to request.
- **Request Headers:** `Content-Type: application/x-www-form-urlencoded`
- **Request Parameters:**
 - `username` (string, required): Admin username (default: `admin`)
 - `password` (string, required): Admin password (default: `admin`)
 - `csrf_token` (string, required): CSRF security token
- **Validation:** Verifies non-empty string and verifies password hash against `AdminUser.password_hash` via `check_password_hash`.
- **Response Success:** 302 Redirect to `/dashboard`. Session variables `session['admin_logged_in'] = True` set.
- **Response Error:** 200 OK rendering `auth/admin_login.html` with banner "Invalid Username or Password".
- **Database Operations:** `AdminUser.query.filter_by(username=username).first()`.

API-004: Passwordless Learner Login

- **HTTP Method:** `POST`
- **Endpoint:** `/learner/login`
- **Purpose:** Log in learners via Global ID, calculate login streaks, and award gamification points.
- **Authentication:** None required.
- **Request Parameters:**
 - `global_id` (string, required): Learner Global ID (e.g. `10001` or alias `learner01`).
 - `class_id` (string, optional): Target class ID to redirect after login.
 - `course_id` (string, optional): Target course ID to redirect after login.
- **Success Flow:** Sets `session['learner_id']`, calculates streak against `last_active_date`, invokes `award_points()`, redirects to `/learners/portal`.
- **Database Operations:** `Learner.query`, updates `current_streak`, `points`, `last_active_date`, inserts `LearnerBadge`.

API-022: Mobile QR Attendance Scanning

- **HTTP Method:** `GET`
- **Endpoint:** `/attendance/scan/<class_id>`
- **Purpose:** Records instantaneous live class attendance when a learner scans a QR code with their mobile device.
- **Authentication:** Required (`session['learner_id']`).
- **Path Parameters:** `class_id` (string/int): Target LiveClass ID.

- **Logic:** Checks if `LiveClass` is locked (`is_locked == True`). If locked, returns error "Class attendance is locked". If open, queries `Attendance`. If record exists, returns "Attendance already recorded". Otherwise creates `Attendance(class_id=..., learner_id=..., status='Present', recorded_via='QR')`.
 - **Database Operations:** Inserts row into `attendances` table.
-

<!-- Database_Documentation.md -->

Learning Hub V3 — Complete Database Documentation & Entity Reference

This document provides complete documentation for all database entities, tables, fields, data types, constraints, foreign keys, cascading behavior, and soft delete/archival policies in **Learning Hub V3**.

1. Relational Data Dictionary

1.1 `admin_users` Table

- **Purpose:** Stores administrator account credentials.
- **Fields:**
 - `id` (Integer, Primary Key, Auto-increment)
 - `username` (String(80), Unique, Not Null, Default: `'admin'`)
 - `password_hash` (String(255), Not Null) — Hashed via PBKDF2/SHA256
 - `name` (String(120), Default: `'L&D Administrator'`)
 - `profile_picture` (String(255), Nullable)
 - `date_of_birth` (Date, Nullable)
 - `created_at` (DateTime, Default: `utcnow`)

1.2 `learners` Table

- **Purpose:** Stores employee and student learner profiles, org structure, streaks, and points.
- **Fields:**
 - `id` (Integer, Primary Key, Auto-increment)
 - `global_id` (String(50), Unique, Not Null, Indexed) — e.g. `'10001'`
 - `name` (String(120), Not Null)
 - `email` (String(120), Nullable)
 - `profile_picture` (String(255), Nullable)
 - `department` (String(100), Default: `'L&D'`)
 - `date_of_birth` (Date, Nullable)

- **designation** (String(120), Nullable)
- **location** (String(120), Nullable) — e.g. 'Hyderabad'
- **branch** (String(120), Nullable) — e.g. 'Madhapur'
- **manager_id** (Integer, Foreign Key **learners.id**, Indexed, Nullable)
- **points** (Integer, Not Null, Default: 0, Indexed)
- **current_streak** (Integer, Not Null, Default: 0)
- **last_active_date** (Date, Nullable)
- **theme** (String(50), Not Null, Default: 'navy')
- **created_at** (DateTime, Default: **utcnow**)
- **Relationships:**
 - **subordinates**: Self-referential backref **manager**
 - **enrollments**: Cascade delete orphan
 - **attendances**: Cascade delete orphan
 - **certificates**: Cascade delete orphan
 - **badges**: Cascade delete orphan

1.3 courses Table

- **Purpose:** Stores self-paced and live course offerings.
- **Fields:**
 - **id** (Integer, Primary Key, Auto-increment)
 - **course_id** (String(20), Unique, Not Null, Indexed) — e.g. 'CRS-SP-001'
 - **name** (String(150), Not Null)
 - **duration_hours** (Float, Default: 1.0)
 - **description** (Text, Nullable)
 - **mode** (String(20), Default: 'Live') — 'Self Paced', 'Live Online', 'Live In Person'
 - **pass_percentage** (Float, Not Null, Default: 80.0)
 - **feedback_repo_id** (Integer, Foreign Key **feedback_repositories.id**, Nullable)
 - **has_certificate** (Boolean, Default: **True**)
 - **thumbnail_filename** (String(255), Nullable)
 - **is_sequential** (Boolean, Default: **True**)
 - **completion_date** (DateTime, Nullable)
 - **is_archived** (Boolean, Default: **False**, Not Null) — Soft deletion flag
 - **created_at** (DateTime, Default: **utcnow**)
 - **pre_quiz_id** (Integer, Foreign Key **quizzes.id**, Nullable)
 - **post_quiz_id** (Integer, Foreign Key **quizzes.id**, Nullable)

1.4 course_lessons Table

- **Purpose:** Stores individual lesson modules belonging to a course.
- **Fields:**
 - `id` (Integer, Primary Key)
 - `course_id` (Integer, Foreign Key `courses.id`, Not Null)
 - `lesson_number` (Integer, Default: 1)
 - `title` (String(150), Not Null)
 - `summary` (Text, Nullable)
 - `content` (Text, Nullable)
 - `video_url` (String(500), Nullable)
 - `duration_hours` (Float, Default: 1.0)
 - `min_time_minutes` (Float, Default: 1.0) — Required minimum time spent before lesson completion
 - `deadline` (DateTime, Nullable)
 - `created_at` (DateTime, Default: `utcnow`)
 - `pre_quiz_id` (Integer, Foreign Key `quizzes.id`, Nullable)
 - `post_quiz_id` (Integer, Foreign Key `quizzes.id`, Nullable)

1.5 lesson_courseware Table

- **Purpose:** Stores multi-format learning objects attached to a lesson.
- **Fields:**
 - `id` (Integer, Primary Key)
 - `lesson_id` (Integer, Foreign Key `course_lessons.id`, Not Null)
 - `title` (String(150), Not Null)
 - `courseware_type` (String(80), Default: 'Text') — 'Video', 'PDF', 'PPT', 'Text', 'SCORM', 'Google Drive'
 - `filename` (String(255), Nullable)
 - `external_url` (String(500), Nullable)
 - `content_text` (Text, Nullable)
 - `uploaded_at` (DateTime, Default: `utcnow`)

1.6 live_classes Table

- **Purpose:** Stores scheduled in-person and online classroom training sessions.
- **Fields:**
 - `id` (Integer, Primary Key)
 - `class_id` (String(30), Unique, Not Null, Indexed) — e.g. 'CRS-CLS-000001'
 - `class_name` (String(150), Not Null)
 - `course_id` (Integer, Foreign Key `courses.id`, Not Null)
 - `class_mode` (String(20), Default: 'In Person') — 'In Person', 'Online'

- `class_date` (Date, Not Null)
- `location` (String(100), Nullable)
- `branch` (String(100), Nullable)
- `session_time` (String(50), Default: 'Morning')
- `meet_link` (String(255), Nullable) — Google Meet URL for online mode
- `facilitator_id` (Integer, Foreign Key `learners.id`, Not Null)
- `co_facilitator_id` (Integer, Foreign Key `learners.id`, Nullable)
- `duration_hours` (Float, Default: 1.0)
- `expected_attendance` (Integer, Default: 30)
- `feedback_repo_id` (Integer, Foreign Key `feedback_repositories.id`, Nullable)
- `quiz_id` (Integer, Foreign Key `quizzes.id`, Nullable)
- `is_locked` (Boolean, Default: False) — Prevents attendance edits when locked
- `locked_at` (DateTime, Nullable)
- `unlock_reason` (Text, Nullable)
- `created_at` (DateTime, Default: `utcnow`)

1.7 learner_enrollments Table

- **Purpose:** Tracks learner course enrollments, current progress, assessment scores, and completion status.
- **Fields:**
 - `id` (Integer, Primary Key)
 - `learner_id` (Integer, Foreign Key `learners.id`, Not Null, Indexed)
 - `course_id` (Integer, Foreign Key `courses.id`, Not Null, Indexed)
 - `class_id` (Integer, Foreign Key `live_classes.id`, Nullable)
 - `completion_status` (String(30), Default: 'Enrolled') — 'Enrolled', 'In Progress', 'Completed', 'Failed'
 - `current_lesson` (Integer, Default: 1)
 - `attempts_count` (Integer, Default: 0)
 - `final_score` (Float, Nullable)
 - `assigned_at` (DateTime, Default: `utcnow`)
 - `completion_date` (DateTime, Nullable)
 - `extended_deadline` (DateTime, Nullable)
 - `extension_requested` (Boolean, Default: False)

1.8 attendances Table

- **Purpose:** Stores live class attendance entries.
- **Fields:**
 - `id` (Integer, Primary Key)
 - `class_id` (Integer, Foreign Key `live_classes.id`, Not Null)

- `learner_id` (Integer, Foreign Key `learners.id`, Not Null)
- `status` (String(20), Default: `'Present'`) — `'Present'`, `'Absent'`, `'Late'`
- `recorded_via` (String(20), Default: `'QR'`) — `'QR'`, `'Manual'`
- `manual_reason` (Text, Nullable)
- `timestamp` (DateTime, Default: `utcnow`)

1.9 certificates Table

- **Purpose:** Stores earned verifiable PDF certificates.
- **Fields:**
 - `id` (Integer, Primary Key)
 - `certificate_id` (String(50), Unique, Not Null, Indexed) — e.g. `'CERT-A1B2C3'`
 - `learner_id` (Integer, Foreign Key `learners.id`, Not Null)
 - `course_id` (Integer, Foreign Key `courses.id`, Not Null)
 - `issue_date` (DateTime, Default: `utcnow`)
 - `pdf_filename` (String(255), Nullable)

1.10 audit_logs Table

- **Purpose:** Compliance log for manual attendance overrides and roster unlocks.
- **Fields:**
 - `id` (Integer, Primary Key)
 - `entity_type` (String(50), Not Null) — e.g. `'LiveClass'`, `'Attendance'`
 - `entity_id` (String(50), Not Null)
 - `action` (String(50), Not Null) — e.g. `'UNLOCK'`, `'MANUAL_ATTENDANCE'`
 - `reason` (Text, Not Null)
 - `performed_by` (String(100), Default: `'admin'`)
 - `timestamp` (DateTime, Default: `utcnow`)

<!-- ER_Diagram.md -->

Learning Hub V3 — Entity Relationship (ER) Diagram

This document renders the complete Entity-Relationship diagram for **Learning Hub V3** using Mermaid ER notation.

```
erDiagram
    AdminUser {
        int id PK
        string username UK
        string password_hash
        string name
```

```
string profile_picture
date date_of_birth
datetime created_at
}

Learner {
int id PK
string global_id UK "Indexed"
string name
string email
string profile_picture
string department
string designation
string location
string branch
int manager_id FK
int points "Indexed"
int current_streak
date last_active_date
string theme
datetime created_at
}

Course {
int id PK
string course_id UK "Indexed"
string name
float duration_hours
string description
string mode
float pass_percentage
int feedback_repo_id FK
boolean has_certificate
string thumbnail_filename
boolean is_sequential
datetime completion_date
boolean is_archived
datetime created_at
int pre_quiz_id FK
int post_quiz_id FK
}

CourseLesson {
int id PK
int course_id FK
int lesson_number
string title
string summary
string content
string video_url
float duration_hours
float min_time_minutes
datetime deadline
datetime created_at
}

LessonCourseware {
int id PK
int lesson_id FK
string title
string courseware_type
string filename
string external_url
text content_text
datetime uploaded_at
}

LiveClass {
int id PK
string class_id UK "Indexed"
```

```
string class_name
int course_id FK
string class_mode
date class_date
string location
string branch
string session_time
string meet_link
int facilitator_id FK
int co_facilitator_id FK
float duration_hours
int expected_attendance
int feedback_repo_id FK
boolean is_locked
datetime locked_at
text unlock_reason
datetime created_at
}

LearnerEnrollment {
int id PK
int learner_id FK "Indexed"
int course_id FK "Indexed"
int class_id FK
string completion_status
int current_lesson
int attempts_count
float final_score
datetime assigned_at
datetime completion_date
datetime extended_deadline
boolean extension_requested
}

Attendance {
int id PK
int class_id FK
int learner_id FK
string status
string recorded_via
text manual_reason
datetime timestamp
}

Certificate {
int id PK
string certificate_id UK "Indexed"
int learner_id FK
int course_id FK
datetime issue_date
string pdf_filename
}

FeedbackRepository {
int id PK
string title
text description
datetime created_at
}

FeedbackQuestion {
int id PK
int repo_id FK
text question_text
string question_type
text options_json
}

FeedbackResponse {
int id PK
```

```

int repo_id FK
int class_id FK
int learner_id FK
text responses_json
datetime submitted_at
}

LearnerBadge {
int id PK
int learner_id FK
string badge_name
string icon
string description
datetime earned_at
}

LmsIssue {
int id PK
int learner_id FK
string category
text description
string status
datetime created_at
datetime resolved_at
}

AuditLog {
int id PK
string entity_type
string entity_id
string action
text reason
string performed_by
datetime timestamp
}

Learner ||--o{ Learner : "manages (subordinates)"
Learner ||--o{ LearnerEnrollment : "has"
Learner ||--o{ Attendance : "records"
Learner ||--o{ Certificate : "earns"
Learner ||--o{ LearnerBadge : "receives"
Learner ||--o{ LmsIssue : "logs"
Course ||--o{ CourseLesson : "contains"
Course ||--o{ LiveClass : "schedules"
Course ||--o{ LearnerEnrollment : "enrolls"
CourseLesson ||--o{ LessonCourseware : "includes"
LiveClass ||--o{ Attendance : "tracks"
LearnerEnrollment ||--o{ Certificate : "generates"
FeedbackRepository ||--o{ FeedbackQuestion : "defines"
FeedbackRepository ||--o{ FeedbackResponse : "collects"

```

<!-- Screen_Documentation.md -->

Learning Hub V3 — UI Screen Documentation Inventory

This document provides a screen-by-screen inventory of all accessible user interface screens in **Learning Hub V3**, documenting URLs, user roles, layout components, forms, tables, modals, states, backend dependencies, and screenshot references.

Screen Inventory Summary Table

Screen ID	Screen Name	Route / URL	Role	Status
SCR-01	Admin Login Screen	/login	Public / Admin	Documented
SCR-02	Learner Login Screen	/learner/login	Public / Learner	Documented
SCR-03	Executive Admin Dashboard	/dashboard	Admin	Documented
SCR-04	Course Management Catalog	/courses/	Admin	Documented
SCR-05	Course Creation & Edit Form	/courses/create	Admin	Documented
SCR-06	Interactive Rise 360 Editor	/courses/rise_editor/<id>	Admin	Documented
SCR-07	SCORM Player Iframe Screen	/courses/courseware/<id>/scorm_player	Learner	Documented
SCR-08	Live Classes Schedule Roster	/classes/	Admin / Facilitator	Documented
SCR-09	Live Class QR Seating Screen	/attendance/qr_view/<id>	Facilitator	Documented
SCR-10	Learner Mobile QR Scanner	/attendance/scan/<id>	Learner	Documented
SCR-11	Manual Attendance & Audit Screen	/attendance/manual/<id>	Admin	Documented
SCR-12	Learner Directory & Import	/learners/	Admin	Documented
SCR-13	Main Learner Portal	/learners/portal	Learner	Documented
SCR-14	Social Learning Community Wall	/learning_wall/	All Users	Documented
SCR-15	Feedback Survey Repository	/feedback/	Admin	Documented
SCR-16	Certificate Download & Verification	/certificates/verify	Public / Learner	Documented
SCR-17	Executive Reports & Analytics	/reports/	Admin	Documented
SCR-18	Super Admin Operations Console	/super_admin/	Super Admin	Documented

Detailed Screen Specifications

SCR-01 — Admin Login Screen

- **Route:** <http://localhost:5000/login>
- **User Role:** Unauthenticated Admin
- **Purpose:** Authenticates L&D administrators.
- **UI Elements:** Centered login card, [Username](#) text input, [Password](#) password input, [Sign In](#) submit button, link to Learner Login.
- **Error States:** Displays red alert banner "Invalid Username or Password".
- **Backend Dependencies:** [auth_bp.admin_login](#), [AdminUser.check_password\(\)](#).

Figure 1 — Admin Login Screen

![Admin Login Screen](../07_UI/Screenshots/admin_login.png)

Screenshot unavailable — requires manual capture.

SCR-02 — Learner Login Screen

- **Route:** <http://localhost:5000/learner/login>
- **User Role:** Unauthenticated Learner
- **Purpose:** Passwordless authentication for employees/students using Global ID.
- **UI Elements:** Hero banner, [Global ID](#) text input (supports [10001](#) or aliases [learner01-learner05](#)), quick login hint pill, [Enter Portal](#) button.
- **Error States:** Displays alert "Learner with Global ID X not found".
- **Backend Dependencies:** [auth_bp.learner_login](#), [gamification.award_points\(\)](#).

Figure 2 — Learner Login Screen

![Learner Login Screen](../07_UI/Screenshots/learner_login.png)

Screenshot unavailable — requires manual capture.

SCR-03 — Executive Admin Dashboard

- **Route:** <http://localhost:5000/dashboard>
- **User Role:** L&D Administrator / Super Admin
- **Purpose:** Central command overview of system activity.
- **UI Elements:** Stat widgets (Total Learners, Active Courses, Live Classes Today, Open Support Tickets), Chart.js enrollment trend graph, Recent System Activity feed, Quick action buttons.
- **Backend Dependencies:** [dashboard_bp.index](#).

Figure 3 — Executive Admin Dashboard

![Admin Dashboard](../07_UI/Screenshots/admin_dashboard.png)

Screenshot unavailable — requires manual capture.

SCR-04 — Course Management Catalog

- **Route:** <http://localhost:5000/courses/>
- **User Role:** L&D Administrator
- **Purpose:** Manages all course offerings.
- **UI Elements:** Filter bar (All, Self Paced, Live Online, Live In Person), search input, course cards with thumbnail, duration badge, mode tag, pass percentage, action buttons ([Manage Lessons](#), [Edit](#), [Archive](#)).
- **Backend Dependencies:** [courses_bp.index](#), [Course.query](#).

Figure 4 — Course Management Catalog

![Course Catalog](../07_UI/Screenshots/courses_catalog.png)

Screenshot unavailable — requires manual capture.

SCR-08 — Live Classes Schedule Roster

- **Route:** <http://localhost:5000/classes/>
- **User Role:** L&D Administrator / Facilitator
- **Purpose:** Schedules live classes and displays session status.
- **UI Elements:** Class schedule table (Class ID, Name, Date, Mode, Facilitator, Roster Count, Lock Status), [Schedule Class](#) button, [QR Display](#) button, [Manual Attendance](#) button.
- **Backend Dependencies:** [classes_bp.index](#), [LiveClass.query](#).

Figure 5 — Live Classes Schedule Roster

![Live Classes](../07_UI/Screenshots/live_classes.png)

Screenshot unavailable — requires manual capture.

SCR-09 — Live Class QR Seating Screen

- **Route:** http://localhost:5000/attendance/qr_view/<class_id>
- **User Role:** Facilitator
- **Purpose:** Projects live session QR code on classroom display screen for student attendance scanning.
- **UI Elements:** High-contrast QR code image, Class ID badge, Facilitator name, live counter of marked learners.
- **Backend Dependencies:** [attendance_bp.qr_view](#), [qr_service.py](#).

Figure 6 — Live Class QR Seating Screen

![QR Seating Screen](../07_UI/Screenshots/qr_seating.png)

Screenshot unavailable — requires manual capture.

SCR-13 — Main Learner Portal

- **Route:** <http://localhost:5000/learners/portal>

- **User Role:** Learner
- **Purpose:** Learner's personalized home dashboard.
- **UI Elements:** Streak flame counter (■ 5 Days), Points counter, Badges showcase, Enrolled Courses carousel, "My Team" tab (for managers), Support ticket shortcut.
- **Backend Dependencies:** `learners_bp.my_portal`, `LearnerEnrollment.query`.

Figure 7 — Main Learner Portal

![Learner Portal](../07_UI/Screenshots/learner_portal.png)

Screenshot unavailable — requires manual capture.

SCR-14 — Social Learning Community Wall

- **Route:** `http://localhost:5000/learning_wall/`
- **User Role:** All Logged-in Users
- **Purpose:** Community feed for announcements, birthday wishes, and peer achievements.
- **UI Elements:** Post creation card, social feed posts, reaction buttons (`like`, `love`, `celebrate`, `clap`, `fire`), comment threads, user profile avatars.
- **Backend Dependencies:** `learning_wall_bp.index`.

Figure 8 — Social Learning Community Wall

![Learning Wall](../07_UI/Screenshots/learning_wall.png)

Screenshot unavailable — requires manual capture.

SCR-18 — Super Admin Operations Console

- **Route:** `http://localhost:5000/super_admin/`
- **User Role:** Super Administrator
- **Purpose:** System-level maintenance and infrastructure management.
- **UI Elements:** DB Backup button, DB Restore file dropzone, System Reset confirmation modal, Storage provider credentials viewer, Audit log table.
- **Backend Dependencies:** `super_admin_bp.index`, `AuditLog.query`.

Figure 9 — Super Admin Operations Console

![Super Admin Console](../07_UI/Screenshots/super_admin.png)

Screenshot unavailable — requires manual capture.

<!-- Authentication_Authorization.md -->

Learning Hub V3 — Security, Authentication & Authorization Audit

This document provides a technical security evaluation of the authentication mechanisms, authorization controls, session management, CSRF protection, and security weaknesses in **Learning Hub V3**.

1. Authentication Mechanisms

1.1 Administrator Authentication

- **Endpoint:** `/login` (POST)
- **Mechanism:** Form-based username/password login.
- **Password Storage & Hashing:** Admin passwords are stored in `admin_users.password_hash` hashed using PBKDF2 with SHA-256 via Werkzeug's `generate_password_hash()` and `check_password_hash()`.
- **Session Assignment:** On successful validation, Flask session variable `session['admin_logged_in'] = True` and `session['admin_username'] = admin.username` are set.

1.2 Learner Authentication

- **Endpoint:** `/learner/login` (POST)
 - **Mechanism:** Passwordless Global ID authentication. Learners submit their corporate/academic Global ID (e.g. `10001` or mapped aliases `learner01-learner05`).
 - **Password Check:** **None.** The current codebase does NOT require or verify a password for learners.
 - **Session Assignment:** On Global ID lookup, system sets `session['learner_id'] = learner.id`, `session['learner_global_id'] = learner.global_id`, `session['learner_name'] = learner.name`, and `session['learner_theme'] = learner.theme`.
 - **Architectural Note in Code:** Inline comments note: *"Later this endpoint will be replaced by Google SSO."*
-

2. Session Management & Cookie Security

- **Session Type:** Flask client-side cryptographically signed session cookies stored in the user's browser.
 - **Secret Key Configuration:** Configured via `SECRET_KEY` environment variable. Defaults to `'narayana-lnd-lms-super-secret-key-2026'` in `app/config.py`.
 - **Session Invalidation:** Endpoint `/logout` clears session data via `session.clear()`.
-

3. Cross-Site Request Forgery (CSRF) Protection

- **CSRF Engine:** Integrated via Flask-WTF `CSRFProtect(app)`.
- **Form Enforcement:** HTML forms in Jinja2 templates include `{{ csrf_token() }}` hidden inputs.

- **CSRF Exemptions:** `b2_bp` blueprint is explicitly exempted from CSRF (`csrf.exempt(b2_bp)` in `app/__init__.py`) to support direct payload uploads to S3 storage endpoints.

4. Protected Routes & Authorization

Route protection is evaluated in route handlers using session variables:

```
# Admin Route Guard Pattern
if not session.get('admin_logged_in'):
    flash('Please log in to access this page.', 'warning')
    return redirect(url_for('auth.admin_login'))

# Learner Route Guard Pattern
learner_id = session.get('learner_id')
if not learner_id:
    flash('Please log in with your Global ID.', 'warning')
    return redirect(url_for('auth.learner_login'))
```

5. Security Audit Findings & Vulnerabilities

[!WARNING]

SEC-01: Passwordless Learner Authentication

Any user who knows or guesses a valid Global ID (e.g., `10001`, `10002`) can log in as that learner without entering a password, accessing their profile, certificates, and team views.

[!WARNING]

SEC-02: Default Hardcoded Administrator Credentials

The database seed script (`app/seed.py`) automatically generates an administrator account with username `admin` and password `admin` if missing.

[!CAUTION]

SEC-03: Hardcoded Fallback Secret Key

If the `SECRET_KEY` environment variable is not defined, Flask falls back to a hardcoded string (`'narayana-1nd-lms-super-secret-key-2026'`), enabling attackers to forge valid session cookies if deployed without `.env`.

[!IMPORTANT]

SEC-04: CSRF Exemption on File Uploads

Disabling CSRF protection on `b2_bp` routes allows cross-site requests to upload unauthenticated payloads to S3 storage endpoints.

<!-- Test_Documentation.md -->

Learning Hub V3 — Testing Strategy & Infrastructure Documentation

This document describes the testing framework, structure, test data, and test coverage assessment of **Learning Hub V3**, based strictly on codebase inspection.

1. Executive Testing Summary

[!CAUTION]

CRITICAL FINDING: 0% AUTOMATED TEST COVERAGE

An exhaustive search of the codebase revealed **NO automated test files** (unit tests, integration tests, API tests, or end-to-end tests) built with frameworks such as [pytest](#), [unittest](#), [robotframework](#), or [playwright](#).

2. Test Infrastructure Audit

Test Layer	Framework Installed	Test Suite Files Found	Execution Status
Unit Tests	None	0 files	Not Implemented
Integration Tests	None	0 files	Not Implemented
API / Endpoint Tests	None	0 files	Not Implemented
UI / E2E Tests	None	0 files	Not Implemented
Performance / Load Tests	None	0 files	Not Implemented

3. Seed-Based Manual Testing Infrastructure

While automated unit tests do not exist, the repository includes a database seed script ([app/seed.py](#)) designed to populate the database with realistic demo data for manual testing:

- **Seed Users:** 1 Admin user ([admin / admin](#)) and 120 Learner records across 7 academic departments and 4 major locations (Hyderabad, Bangalore, Chennai, Pune).
- **Guaranteed Learner:** Global ID [10001](#) (Rajesh Kumar, Academic Director) pre-seeded with 150 gamification points and a 5-day streak.
- **Pre-populated Courses:** 10 Self-Paced courses with YouTube video embeds, 3 Live Online courses (Google Meet links), 3 Live In-Person campus courses.
- **Pre-populated Feedback & Tickets:** 1 standard L&D feedback survey and 3 support tickets ([LmsIssue](#)).

4. Recommendations for Test Implementation

1. **Unit Testing Framework:** Introduce [pytest](#) and [pytest-flask](#) to write unit tests for [assessment_service.py](#), [gamification.py](#), [pdf_service.py](#), [scorm_service.py](#), and [b2_service.py](#).
2. **API Integration Testing:** Implement Flask client integration tests ([app.test_client\(\)](#)) covering authentication endpoints, course CRUD, QR attendance scans, and feedback submission.

3. **End-to-End UI Automation:** Build a Playwright Python test suite to validate Learner Login, SCORM playback, QR attendance scanning, and certificate download.

<!-- Test_Coverage_Matrix.md -->

Learning Hub V3 — Test Coverage Matrix

This matrix documents the automated test coverage status across all feature modules in **Learning Hub V3**.

Module-by-Module Test Coverage Matrix

Module / Blueprint	Feature	Unit Test Exists	Integration Test Exists	E2E Test Exists	Overall Coverage
auth_bp	Admin Login / Logout	■ No	■ No	■ No	0% (UNTESTED)
auth_bp	Learner Passwordless Login	■ No	■ No	■ No	0% (UNTESTED)
courses_bp	Course Creation & Management	■ No	■ No	■ No	0% (UNTESTED)
courses_bp	SCORM Package Player	■ No	■ No	■ No	0% (UNTESTED)
courses_bp	Rise 360 Block Authoring	■ No	■ No	■ No	0% (UNTESTED)
classes_bp	Live Class Scheduling	■ No	■ No	■ No	0% (UNTESTED)
attendance_bp	Real-Time QR Code Scanning	■ No	■ No	■ No	0% (UNTESTED)
attendance_bp	Manual Attendance & Audit Log	■ No	■ No	■ No	0% (UNTESTED)
certificates_bp	ReportLab PDF Certificate Gen	■ No	■ No	■ No	0% (UNTESTED)
certificates_bp	Certificate Verification	■ No	■ No	■ No	0% (UNTESTED)
feedback_bp	Feedback Survey Engine	■ No	■ No	■ No	0% (UNTESTED)
learning_wall_bp	Social Wall Posts & Reactions	■ No	■ No	■ No	0% (UNTESTED)

learners_bp	Learner Directory & Excel Import	■ No	■ No	■ No	0% (UNTESTED)
learners_bp	Support Ticket Helpdesk	■ No	■ No	■ No	0% (UNTESTED)
super_admin_bp	DB Backup / Restore / Reset	■ No	■ No	■ No	0% (UNTESTED)
b2_bp	S3 / Backblaze B2 Upload	■ No	■ No	■ No	0% (UNTESTED)

Untested Critical Functionality Risk Assessment

[!WARNING]

High-risk critical paths currently lacking automated test regression coverage:

1. **Assessment Grading & Certification Trigger:** Incorrect pass percentage calculations could issue certificates to failing learners.

2. **Audit Logging Integrity:** Manual attendance overrides must be guaranteed to log audit entries without silent failures.

3. **SCORM Unzipping & Path Traversal:** Untested SCORM zip extraction could lead to directory traversal security risks.

4. **Excel Import Parsing:** Malformed Excel rows during learner import could corrupt database tables without schema validation tests.

<!-- Deployment_Guide.md -->

Learning Hub V3 — Deployment & Environment Setup Guide

This guide provides step-by-step instructions to configure, initialize, and deploy **Learning Hub V3** in local development, production Waitress/Gunicorn environments, and cloud serverless environments (Vercel/Render).

1. Prerequisites & Runtime Requirements

- **Python Version:** Python 3.10+ (tested on Python 3.14.2)
- **Database Backend:** SQLite 3 (Development) or PostgreSQL 13+ (Production)
- **C Compiler / Build Tools:** Required for `psycpg2-binary`, `pypdfium2`, `reportlab`, `pillow`
- **Object Storage (Optional):** AWS S3, Backblaze B2, or self-hosted MinIO bucket for decoupled file hosting

2. Environment Configuration (.env)

Create a `.env` file in the root directory `LearningHubV3/` based on `.env.example`:

```
# Flask Core Configuration
SECRET_KEY=<configured_secure_random_key>
FLASK_ENV=production
PORT=5000

# Database Configuration (SQLite default fallback; PostgreSQL for Production scale)
DATABASE_URL=postgresql://<user>:<password>@<hostname>:5432/<dbname>

# Decoupled Object Storage Provider Configuration ('local' or 's3')
STORAGE_PROVIDER=s3
S3_ACCESS_KEY=<configured_s3_key_id>
S3_SECRET_KEY=<configured_s3_application_key>
S3_ENDPOINT_URL=https://s3.us-west-004.backblazeb2.com
S3_BUCKET=narayana-lms

# Feature Flags
ENABLE_CONTENT_AUTHORING=True
```

[!CAUTION]

NEVER commit actual API keys, credentials, or production database connection strings to version control.

3. Local Development Setup

```
# 1. Navigate to product directory
cd LearningHubV3

# 2. Create Python virtual environment
python -m venv venv

# 3. Activate virtual environment (Windows PowerShell)
.\venv\Scripts\Activate.ps1

# 4. Install dependencies
pip install -r requirements.txt

# 5. Initialize database tables and seed initial demo data
python -c "from app import create_app; from app.seed import init_db_and_seed; app=create_app();
init_db_and_seed(app)"

# 6. Run development server
python run.py
```

App will be accessible at <http://localhost:5000>.

4. Production Deployment

A. Production Deployment on Windows/Linux (Waitress WSGI)

`run.py` detects `FLASK_ENV=production` and automatically starts the Waitress WSGI server:

```
python run.py
```

B. Production Deployment on Render / Linux (Gunicorn)

Use the included `Procfile`:

```
web: gunicorn run:app --bind 0.0.0.0:$PORT --timeout 120
```

C. Serverless Deployment on Vercel

Configuration is pre-built via `vercel.json` and `api/index.py`:

```
{
  "version": 2,
  "builds": [{ "src": "api/index.py", "use": "@vercel/python" }],
  "routes": [{ "src": "/(.*)", "dest": "api/index.py" }]
}
```

5. PostgreSQL Database Migration

To transition from local SQLite to PostgreSQL for 60,000+ active learners:

```
# 1. Update DATABASE_URL in .env
DATABASE_URL=postgresql://lms_user:secure_password@localhost:5432/lms_db

# 2. Run Flask-Migrate dynamic upgrade
flask db upgrade
```

<!-- Operations_Runbook.md -->

Learning Hub V3 — Operations & Maintenance Runbook

This runbook provides system administrators and L&D operations teams with blueprints to monitor, maintain, backup, troubleshoot, and scale **Learning Hub V3**.

1. System Control Commands

Start System Server (Development)

```
python run.py
```

Start System Server (Production - Waitress)

```
set FLASK_ENV=production
python run.py
```

Stop System Server

Press `Ctrl + C` in console or terminate WSGI process (`taskkill /F /IM python.exe` on Windows).

2. Database Backup & Disaster Recovery

2.1 Database Backup via Super Admin Web Console

1. Log in as Super Admin (`/login`).

2. Navigate to http://localhost:5000/super_admin/.
3. Click **"Download Database Backup"**.
4. System serves a downloadable archive of `lms.db`.

2.2 Manual File Backup (SQLite)

```
cp lms.db backups/lms_backup_$(date +%Y%m%d_%H%M%S).db
```

2.3 Database Disaster Recovery (Restore)

1. Stop Flask WSGI server.
 2. Replace corrupt `lms.db` with latest backup file.
 3. Restart server (`python run.py`).
-

3. High-Scale Storage Decoupling Blueprint (MinIO & Backblaze B2)

To prevent server storage exhaustion when streaming video and SCORM packages to 60,000+ learners:

Self-Hosted MinIO Container Setup

```
docker run -d \  
-p 9000:9000 \  
-p 9001:9001 \  
--name minio \  
-v /mnt/data:/data \  
-e "MINIO_ROOT_USER=admin" \  
-e "MINIO_ROOT_PASSWORD=<configured_secure_password>" \  
minio/minio server /data --console-address ":9001"
```

Configure `.env`:

```
STORAGE_PROVIDER=s3  
S3_ACCESS_KEY=admin  
S3_SECRET_KEY=<configured_secure_password>  
S3_ENDPOINT_URL=http://localhost:9000  
S3_BUCKET=narayana-lms
```

4. Log Inspection & Monitoring

- **Flask Console Output:** Log messages are output directly to stdout/stderr.
 - **Audit Logs Table:** All administrative roster unlocks and manual attendance updates are stored in table `audit_logs` and viewable under http://localhost:5000/super_admin/.
-

<!-- Troubleshooting.md -->

Learning Hub V3 — Operational Troubleshooting Guide

This document lists common operational issues, error codes, root causes, and step-by-step resolution procedures for **Learning Hub V3**.

Troubleshooting Decision Matrix

Issue Code	Symptoms	Root Cause	Resolution Procedure
ERR-01	<code>sqlite3.OperationalError: database is locked</code>	High concurrent write transactions locking SQLite file.	Ensure SQLite is running in WAL mode (<code>PRAGMA journal_mode=WAL</code>). For >500 active concurrent users, migrate to PostgreSQL by updating <code>DATABASE_URL</code> in <code>.env</code> .
ERR-02	HTTP 413 "The uploaded file is too large"	Uploaded file size exceeds 1 GB limit.	Check uploaded file size. If larger than 1 GB, compress video or upload via S3/B2 storage endpoint directly.
ERR-03	SCORM package fails to load inside iframe	Missing <code>imsmanifest.xml</code> or corrupted ZIP structure.	Verify SCORM package ZIP contains valid <code>imsmanifest.xml</code> at root directory. Inspect <code>scorm_service.py</code> logs.
ERR-04	QR Attendance scan returns "Class attendance is locked"	Facilitator or admin locked the class roster.	Admin must navigate to <code>/classes/</code> , click Unlock Class, and enter mandatory justification reason.
ERR-05	PDF Certificate fails to generate / download	Missing ReportLab library or template font error.	Verify <code>reportlab</code> is installed in virtual environment (<code>pip install reportlab</code>). Check <code>pdf_service.py</code> console stack trace.
ERR-06	B2 / S3 upload fails with 403 Forbidden	Expired or incorrect S3 keys in <code>.env</code> .	Check <code>S3_ACCESS_KEY</code> and <code>S3_SECRET_KEY</code> in <code>.env</code> . Verify bucket permissions in Backblaze B2 or MinIO console.
ERR-07	Learner cannot log in (Global ID not found)	Global ID typo or learner profile not imported.	Admin navigates to <code>/learners/</code> , verifies learner exists, or imports learner via Excel bulk upload.

<!-- Known_Issues.md -->

Learning Hub V3 — Known Issues Inventory

This document documents all identified bugs, edge-case failures, and implementation anomalies discovered during code inspection of **Learning Hub V3**.

Known Issues Inventory Matrix

Issue ID	Description	Location	Severity	Impact	Current Status	Suggested Resolution
ISS-01	Passwordless Learner Authentication	app/routes/auth.py (L42-151)	High	Any user entering a valid Global ID (e.g. 10001) instantly accesses learner portal without authentication.	Present in Code	Implement password hashing or integrate Google OAuth2 / SSO as indicated in code comments.
ISS-02	Hardcoded Admin Credentials in Seed Script	app/seed.py (L116-118)	High	Seeding generates admin user admin with password admin .	Present in Code	Enforce environment-variable driven initial admin password setup.
ISS-03	CSRF Exemption on B2 Upload Routes	app/__init__.py (L50)	Medium	b2_bp is explicitly exempted from CSRF protection (csrf.exempt(b2_bp)).	Present in Code	Implement API token header validation for S3 upload endpoints.
ISS-04	Hardcoded Fallback Secret Key	app/config.py (L9)	Medium	Session secret falls back to static hardcoded string if .env is missing.	Present in Code	Raise explicit error on startup if SECRET_KEY is not provided in production.
ISS-05	Legacy Redundant Columns in live_classes	app/models/live_class.py (L29-30)	Low	Columns facilitator_name and co_facilitator_name stored alongside relational IDs facilitator_id .	Present in Code	Perform DB migration to drop legacy string columns and rely on relational joins.
ISS-06	Lack of Automated Test Suite	Base repository	High	0% test coverage across all 13 Blueprints and core services.	Present in Code	Build unit and integration test suite with pytest .

<!-- Technical_Debt.md -->

Learning Hub V3 — Technical Debt Assessment

This document identifies architectural concerns, missing validation, hardcoded values, deprecated patterns, and maintainability concerns in **Learning Hub V3**.

1. Architectural & Code Quality Assessment

1.1 Large Controller Files

- **Concern:** `app/routes/courses.py` contains **100,532 bytes** of code across a single file handling course CRUD, lesson editing, Rise 360 block parsing, SCORM unzipping, assessment evaluation, and certificate triggers.
- **Impact:** Poor separation of concerns; high risk of merge conflicts and difficulty in maintaining route handlers.
- **Recommendation:** Refactor `courses.py` into separate blueprint controllers (`course_crud.py`, `courseware.py`, `scorm.py`, `rise.py`).

1.2 `app/routes/learners.py` Bloat

- **Concern:** `app/routes/learners.py` contains **70,614 bytes** handling learner profiles, Excel import/export via pandas, team management, external certificate uploads, and support ticketing.
- **Recommendation:** Extract Excel import/export logic into `app/services/excel_service.py` and ticket desk into `app/routes/issues.py`.

1.3 Schema Modification Logic in `app/seed.py`

- **Concern:** `app/seed.py` contains 28 inline raw SQL `ALTER TABLE` statements (L23-52) executed on every app startup wrapped in try/except blocks to bypass missing column errors.
 - **Impact:** Non-standard database schema migration practice bypassing Alembic tracking.
 - **Recommendation:** Remove raw alter statements from seed code and rely strictly on standard `flask db migrate` and `flask db upgrade`.
-

2. Hardcoded Values & Magic Constants

1. **Default Pass Percentage:** Hardcoded to 80% across models (`Course`, `Quiz`) and seed functions.
 2. **Maximum Assessment Attempts:** Hardcoded to 3 attempts in `LearnerEnrollment.attempts_count`.
 3. **Hardcoded Upload Paths:** Direct path joins scattered across routes instead of centralized path helpers.
-

<!-- Requirements_Traceability_Matrix.md -->

Learning Hub V3 — Requirements Traceability Matrix (RTM)

This matrix maps product requirements to features, UI screens, API endpoints, backend logic handlers, database models, test cases, and current implementation status.

Requirement ID	Requirement Summary	Feature ID	UI Screen	API Endpoint	Backend Logic	Database Models	Test Case	Status
FR-001	Admin Authentication	FEAT-01	SCR-01 (/login)	POST /login	auth_bp.admin_login	AdminUser	Untested	Implemented
FR-002	Learner Passwordless Login	FEAT-02	SCR-02 (/learner/login)	POST /learner/login	auth_bp.learner_login	Learner, LearnerBadge	Untested	Implemented
FR-003	Course Management	FEAT-03	SCR-04 (/courses/)	POST /courses/create	courses_bp.create_course	Course	Untested	Implemented
FR-004	Multi-Format Courseware	FEAT-04	SCR-05 (/courses/<id>/lessons)	POST /courses/<id>/lessons/create	courses_bp.add_lesson	CourseLesson, LessonCourseware	Untested	Implemented
FR-005	Rise 360 Authoring	FEAT-06	SCR-06 (/courses/rise_editor)	POST /courses/rise_editor/<id>/save	courses_bp.save_rise	RiseCoursewareVersion	Untested	Implemented
FR-006	SCORM Player	FEAT-05	SCR-07 (/courses/scorm_player)	GET /courses/courseware/<id>/scorm_player	scorm_service.py	LessonCourseware	Untested	Implemented
FR-007	Live Class Scheduling	FEAT-07	SCR-08 (/classes/)	POST /classes/create	classes_bp.create_class	LiveClass	Untested	Implemented
FR-008	Real-Time QR Attendance	FEAT-08	SCR-09, SCR-10	GET /attendance/scan/<id>	attendance_bp.scan	Attendance	Untested	Implemented
FR-009	Attendance Audit Log	FEAT-09	SCR-11 (/attendance/manual)	POST /attendance/manual/<id>	attendance_bp.manual	Attendance, AuditLog	Untested	Implemented
FR-010	Assessment Engine	FEAT-10	Assessment View	POST /courses/<id>/assessment	assessment_service.py	CourseAssessment, AssessmentAttempt	Untested	Implemented

FR-011	PDF Certificate Gen	FEAT-11	Learner Portal	GET /certificates/download/<id>	pdf_service.py	Certificate	Untested	Implemented
FR-012	Cert Verification	FEAT-12	SCR-16 (/certificates/verify)	GET /certificates/verify	certificates_bp.verify	Certificate	Untested	Implemented
FR-013	Feedback Repository	FEAT-13	SCR-15 (/feedback/)	POST /feedback/submit/<id>	feedback_bp.submit	FeedbackRepository, FeedbackResponse	Untested	Implemented
FR-014	Gamification Engine	FEAT-14	SCR-13 (/learners/portal)	Session Hook	gamification.py	Learner, LearnerBadge	Untested	Implemented
FR-015	Social Learning Wall	FEAT-15	SCR-14 (/learning_wall/)	POST /learning_wall/post	learning_wall_bp.index	LearningWallPost, Reaction, Comment	Untested	Implemented
FR-016	Support Ticketing	FEAT-18	Learner Portal	POST /learners/issues/create	learners_bp.issues	LmsIssue	Untested	Implemented
FR-017	Learner Excel Import	FEAT-16	SCR-12 (/learners/)	POST /learners/import	learners_bp.import_excel	Learner	Untested	Implemented
FR-018	Manager Team View	FEAT-17	SCR-13 (My Team)	GET /learners/portal	learners_bp.my_portal	Learner.subordinates	Untested	Implemented
FR-019	Decoupled S3 Storage	FEAT-19	Backend S3	POST /b2/upload	b2_service.py	Cloud Storage Key	Untested	Implemented
FR-020	Super Admin Console	FEAT-20	SCR-18 (/super_admin/)	GET /super_admin/backup_db	super_admin_bp.index	AuditLog, Database file	Untested	Implemented

<!-- DOCUMENTATION_ANALYSIS_REPORT.md -->

DOCUMENTATION ANALYSIS REPORT — LEARNING HUB V3

Date: September 11, 2026

Application Analyzed: Learning Hub V3

Analysis Scope: Complete Codebase, Models, Blueprints, Services, Templates, Configuration, and Live Server Execution.

1. Documentation Generated

A complete professional Software Product Documentation Suite comprising **22 Markdown documents** has been generated under [Documentation/](#):

```
Documentation/
■■■ 01_Product_Vision/Product_Vision.md
■■■ 02_Requirements/SRS.md
■■■ 03_Product/Feature_Catalog.md, User_Roles.md, User_Flows.md
■■■ 04_Architecture/System_Architecture.md, Frontend_Architecture.md, Backend_Architecture.md,
Database_Architecture.md
■■■ 05_API/API_Documentation.md
■■■ 06_Database/Database_Documentation.md, ER_Diagram.md
■■■ 07_UI/Screen_Documentation.md
■■■ 08_Security/Authentication_Authorization.md
■■■ 09_Testing/Test_Documentation.md, Test_Coverage_Matrix.md
■■■ 10_Deployment/Deployment_Guide.md, Operations_Runbook.md
■■■ 11_Maintenance/Troubleshooting.md
■■■ 12_Issues/Known_Issues.md, Technical_Debt.md
■■■ 13_Traceability/Requirements_Traceability_Matrix.md
■■■ 14_As_Built/As_Built_Product_Documentation.md
■■■ DOCUMENTATION_ANALYSIS_REPORT.md
```

2. Codebase Coverage

100% of the active repository was analyzed:

- **Backend Code:** 13 Blueprints in [app/routes/](#), 10 service modules in [app/services/](#), utility modules in [app/utls/](#).
- **Database Models:** All 14 SQLAlchemy ORM model files in [app/models/](#).
- **Frontend Assets:** 14 Jinja2 template directories in [app/templates/](#), [base.html](#), and CSS/JS assets in [app/static/](#).
- **Configuration & Infrastructure:** [run.py](#), [app/config.py](#), [.env.example](#), [Procfile](#), [vercel.json](#), [requirements.txt](#), [SUPER_ADMIN_MANUAL.md](#).

3. Missing Information

The following details could not be determined strictly from code inspection:

- Original product requirement specification documents (BRD / PRD) — *Reconstructed from implementation*.
- Historical SLA performance targets (e.g. max page response time in ms under load).
- Third-party SSO provider client IDs / secrets (commented out as future phase).

4. Documentation Confidence Assessment

Documentation Area	Confidence Level	Justification
--------------------	------------------	---------------

System Architecture	High	Derived directly from active application code, blueprints, and service imports.
API Inventory	High	40 endpoints extracted and mapped directly from Flask blueprint route decorators.
Database Schema	High	Extracted from explicit SQLAlchemy <code>db.Column</code> declarations and ORM relationships.
Security & Auth	High	Verified directly against route authentication guards and seed user creation.
Testing Status	High	Verified 0 automated test files exist across workspace.
Original Product Vision	Medium	Reconstructed based on implemented user flows, role structures, and seed data.

5. Implementation Gaps Discovered

- 1. **Learner Password Protection:** Learner login is passwordless (Global ID lookup only).
- 2. **Automated Unit & E2E Testing:** 0% test coverage across all modules.
- 3. **CSRF Exemption on S3 Uploads:** B2 routes explicitly exempted from CSRF validation.

6. Technical Risks Summary

- 1. **Unauthorized Learner Access:** Passwordless Global ID login allows account spoofing if an employee ID is known.
- 2. **Lack of Automated Testing:** Code modifications risk introducing undetected regressions in grading or certification logic.
- 3. **Monolithic Controller Files:** `courses.py` (100 KB) and `learners.py` (70 KB) create high maintenance complexity.

7. Next Recommended Documentation Steps

- 1. Obtain business SLAs and compliance standards from the product owner.
- 2. Maintain and update this documentation suite upon introducing new features or security patches.